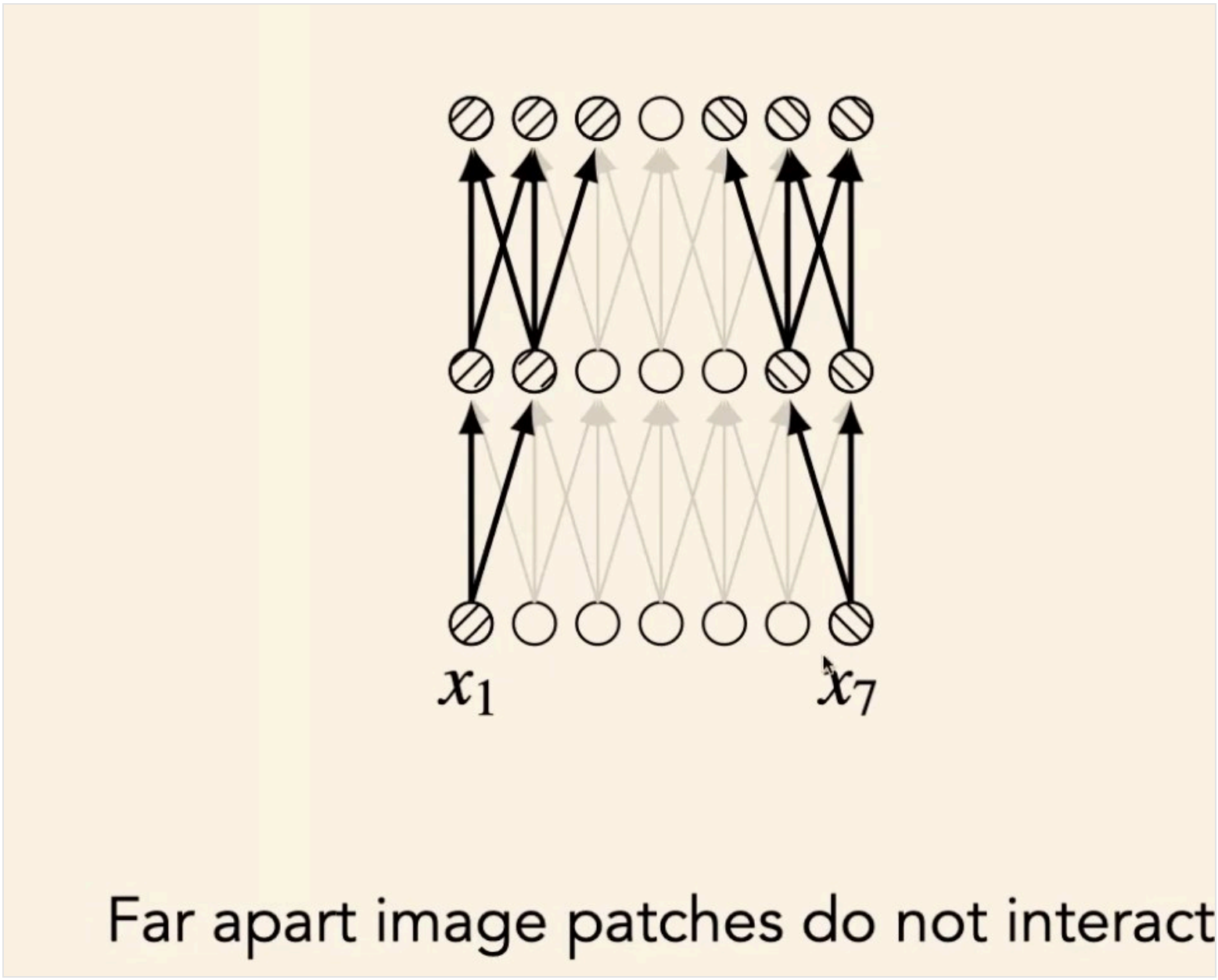
Lec. transformer

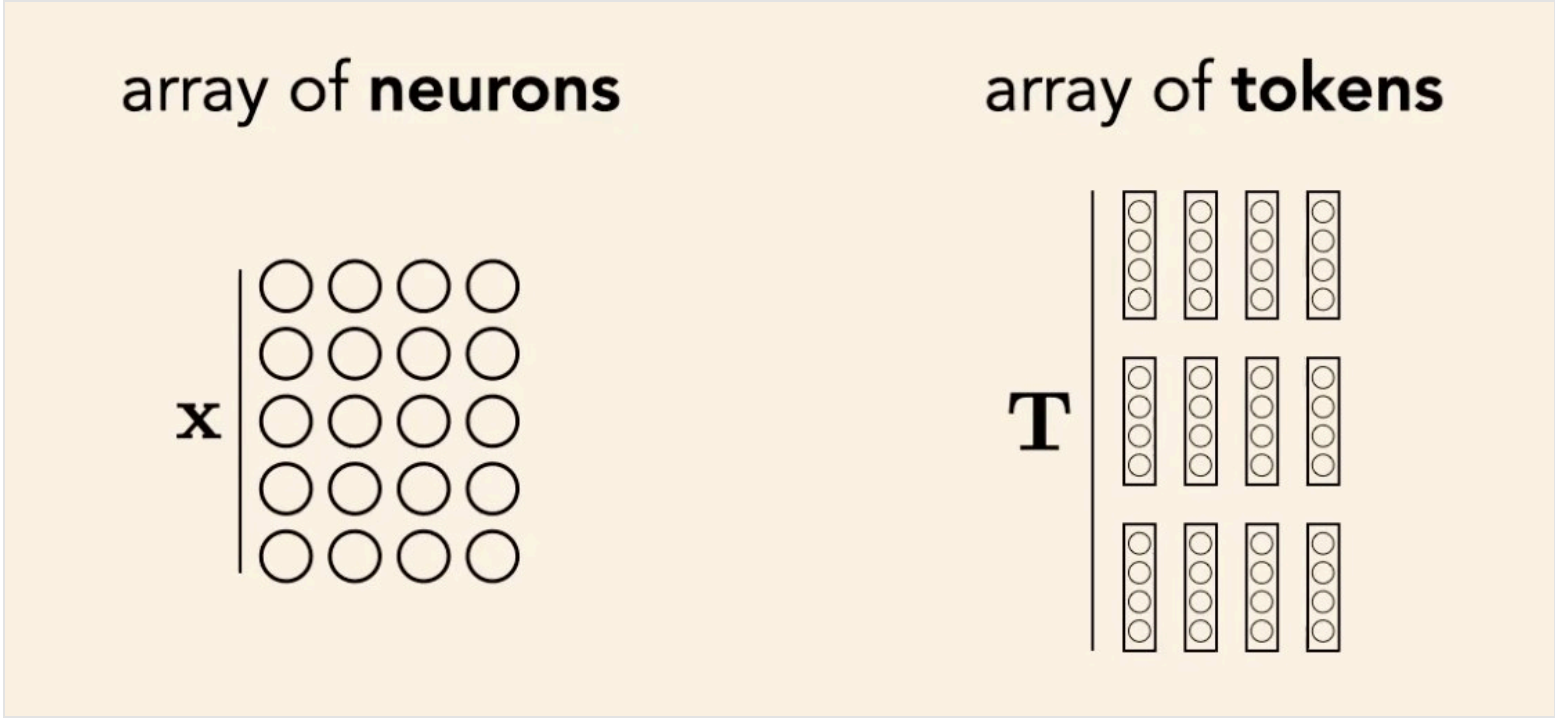
- CNN의 단점



- 멀리있는 패치 → 영향을 주지 않음.
- 계산도 많음.
- → attention을 써서, 문제와 관련된 것에 집중

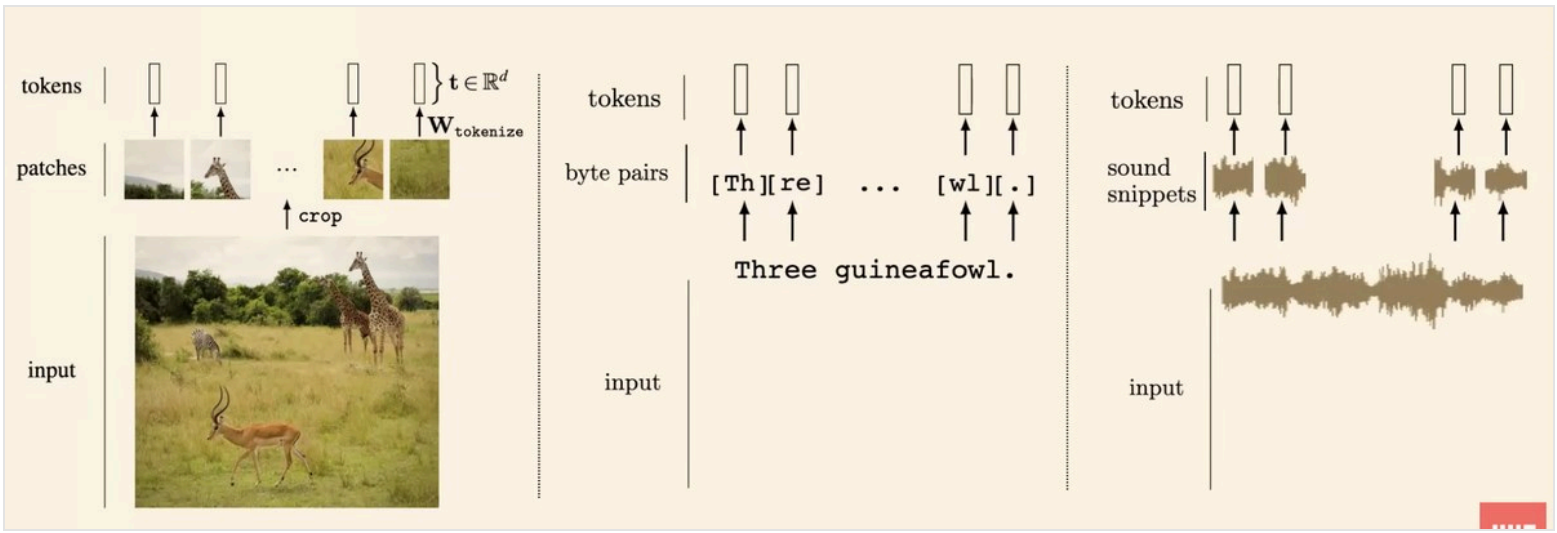
1 tokens

- = 뉴런의 벡터
- array of neurons → array of tokens로 생각해야 함.
  (NLP에서 token은 단어같은 거)
- 신경망 - 스칼라 값을 같는 뉴런들이 있음
  토큰 신경망 - 벡터 값을 같는 유닛들로 구성되어 있음



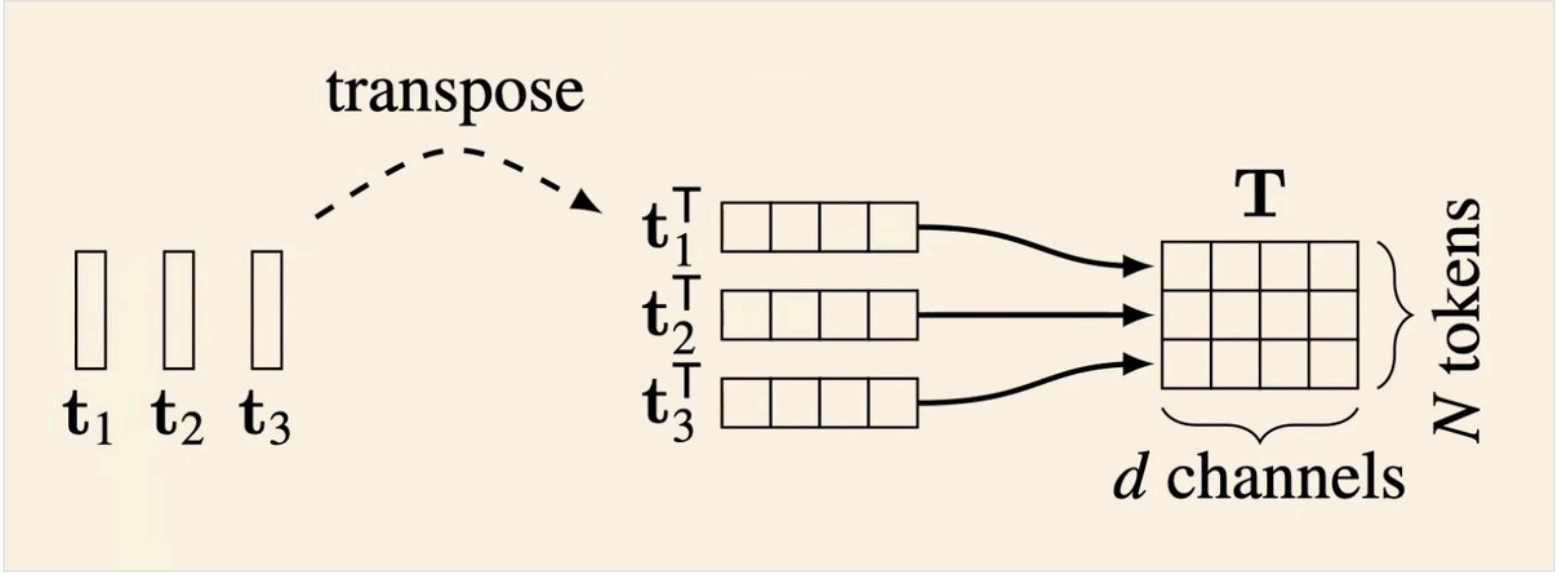
- 여기서도 컨볼루션 같은 걸 할 수 있음

- Tokenizing the input data
- data → token으로 바꿔야 함
- for image. 패치로 분할 → W로 tokenize



- input → byte pairs → tokens 순으로 토큰화

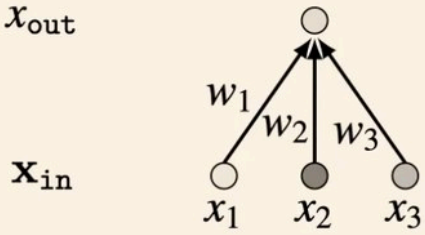
- Notation



- token들을 →matrix T로 만듦.

- Linear combination of tokens

Linear combination of neurons

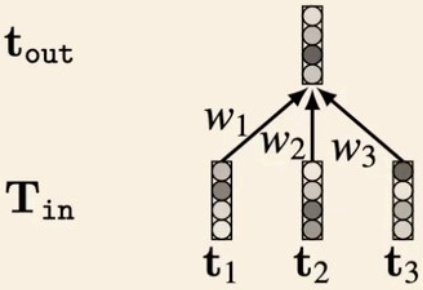


$$x_{out} = w_1 x_1 + w_2 x_2 + w_3 x_3$$

$$x_{out}[i] = \sum_{j=1}^{N} w_{ij} x_{in}[j]$$

$$\mathbf{x}_{out} = \mathbf{W}\mathbf{x}_{in}$$

Linear combination of tokens



$$t_{out} = w_1 t_1 + w_2 t_2 + w_3 t_3$$

$$\mathbf{T}_{out}[i,:] = \sum_{j=1}^{N} w_{ij} \mathbf{T}_{in}[j,:]$$

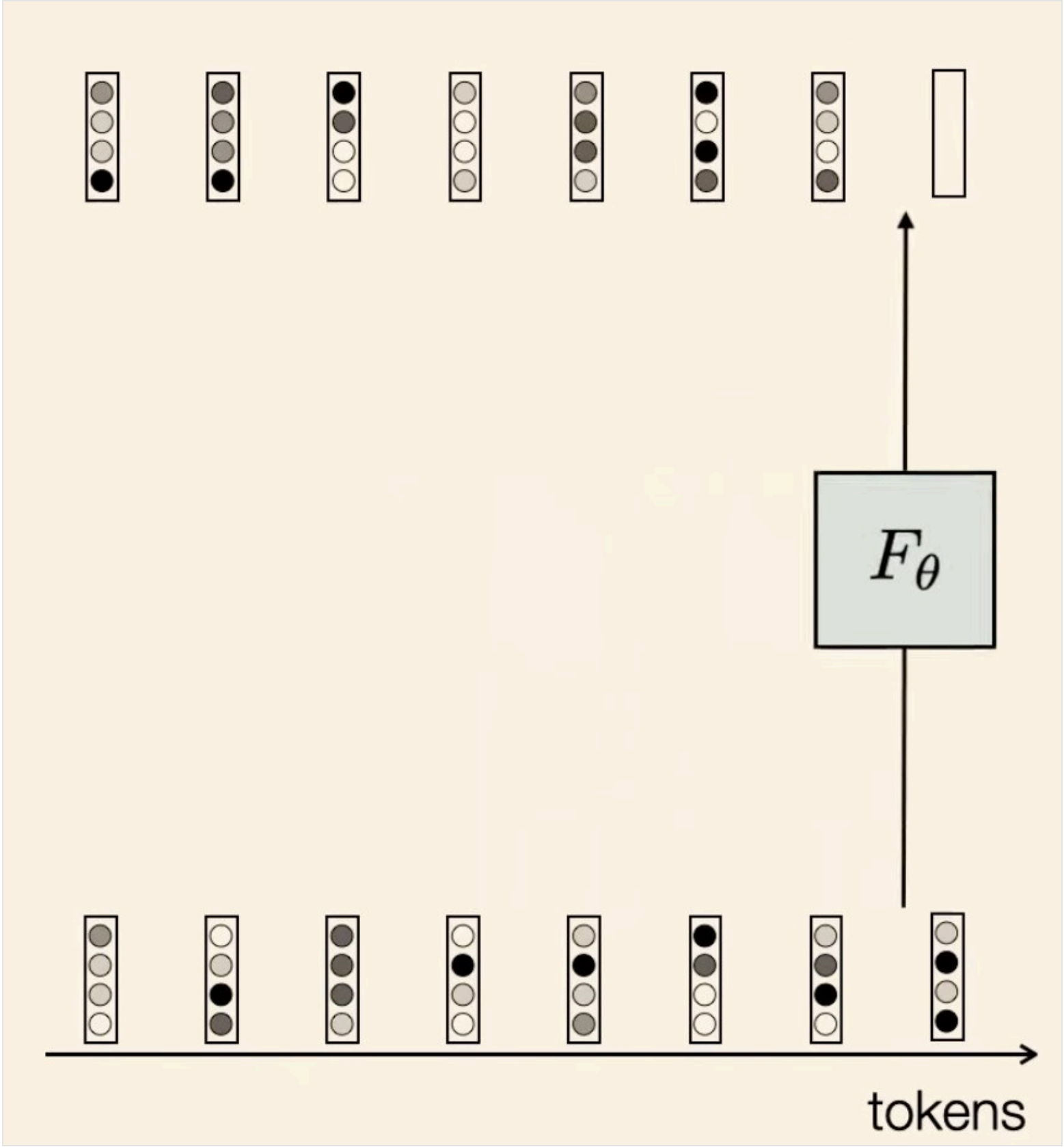
$$\mathbf{T}_{out} = \mathbf{W}\mathbf{T}_{in}$$

- Token-wise nonlinearity

$$\mathbf{x}_{\text{out}} = \begin{bmatrix} \texttt{relu}(x_{\text{in}}[0]) \\ \vdots \\ \texttt{relu}(x_{\text{in}}[N-1]) \end{bmatrix}$$

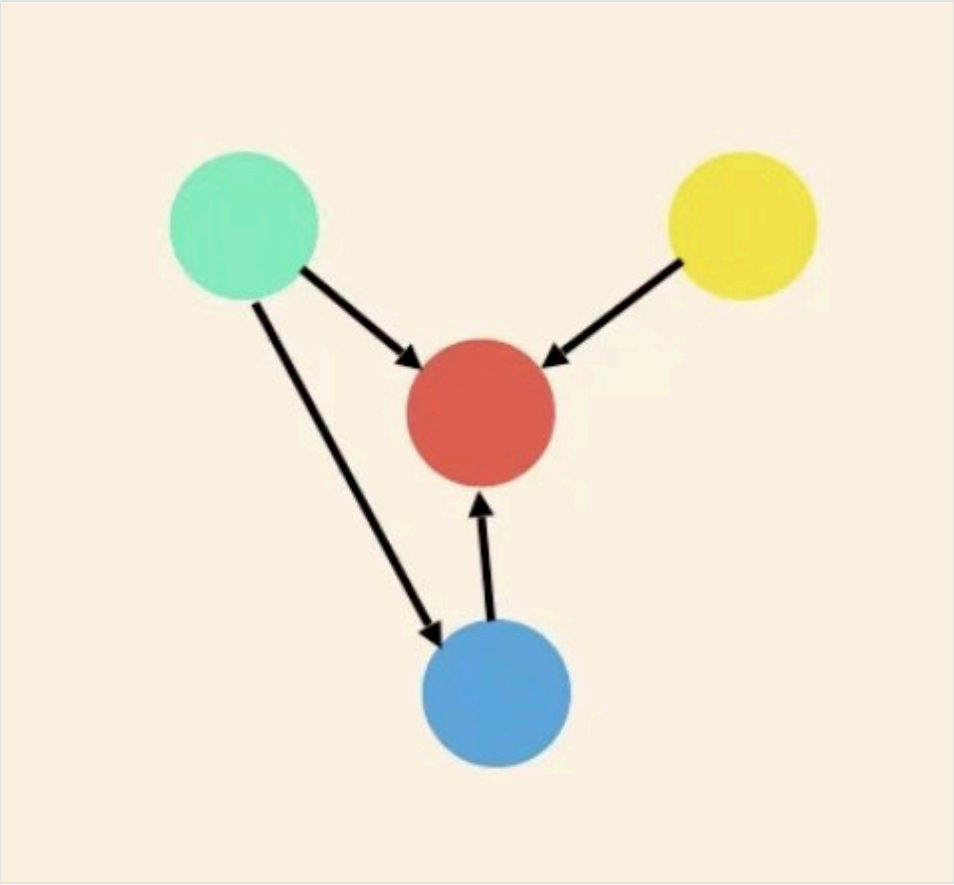
$$\mathbf{T}_{\text{out}} = \begin{bmatrix} F_\theta(\mathbf{T}_{\text{in}}[0,:]) \\ \vdots \\ F_\theta(\mathbf{T}_{\text{in}}[N-1,:]) \end{bmatrix}$$

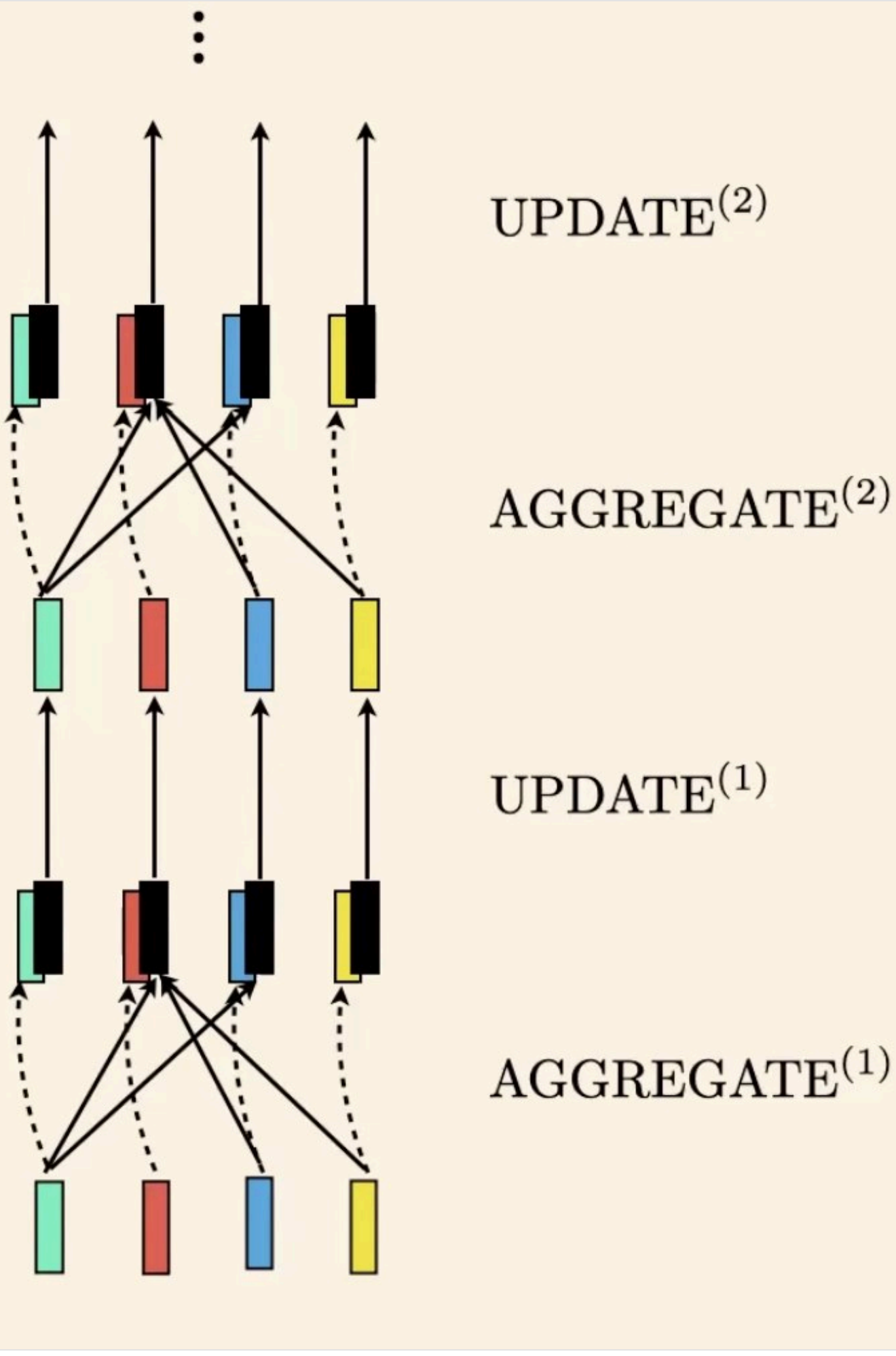
- F를 1x1 컨볼루션 처럼 보아도 됨.



- ReLU처럼 nonlinearity를 만들던 역할을 token에서는 F가 대신함.

- GNN
- 화살표 방향에 있는 거에 영향을 주며 update 되는 구조임

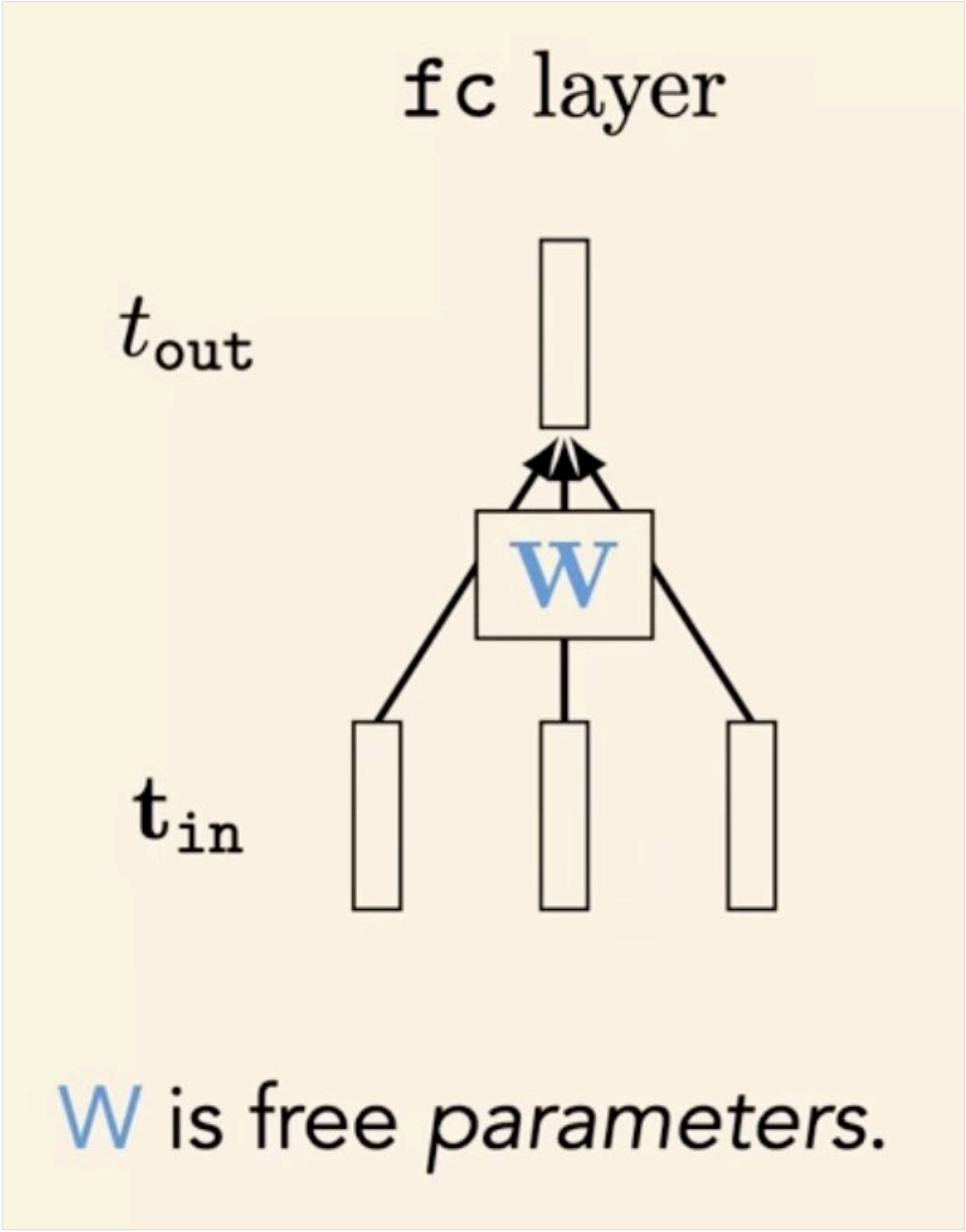



UPDATE(2)
AGGREGATE(2)
UPDATE(1)
AGGREGATE(1)

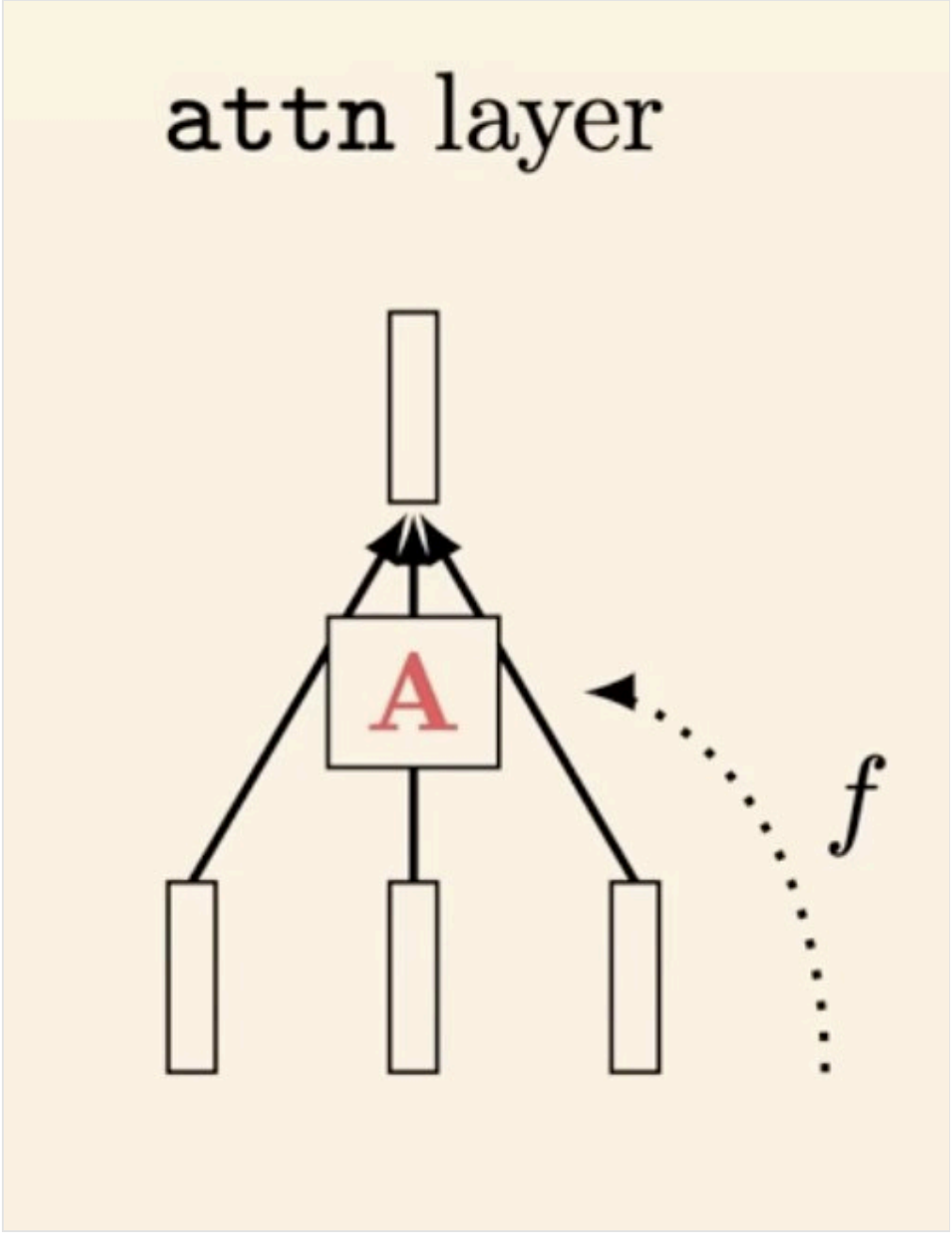
- 트랜스포머는 fully-connected graphs로 볼 수 있음

---

2 attention



- 원래 토큰이 학습되는 방식



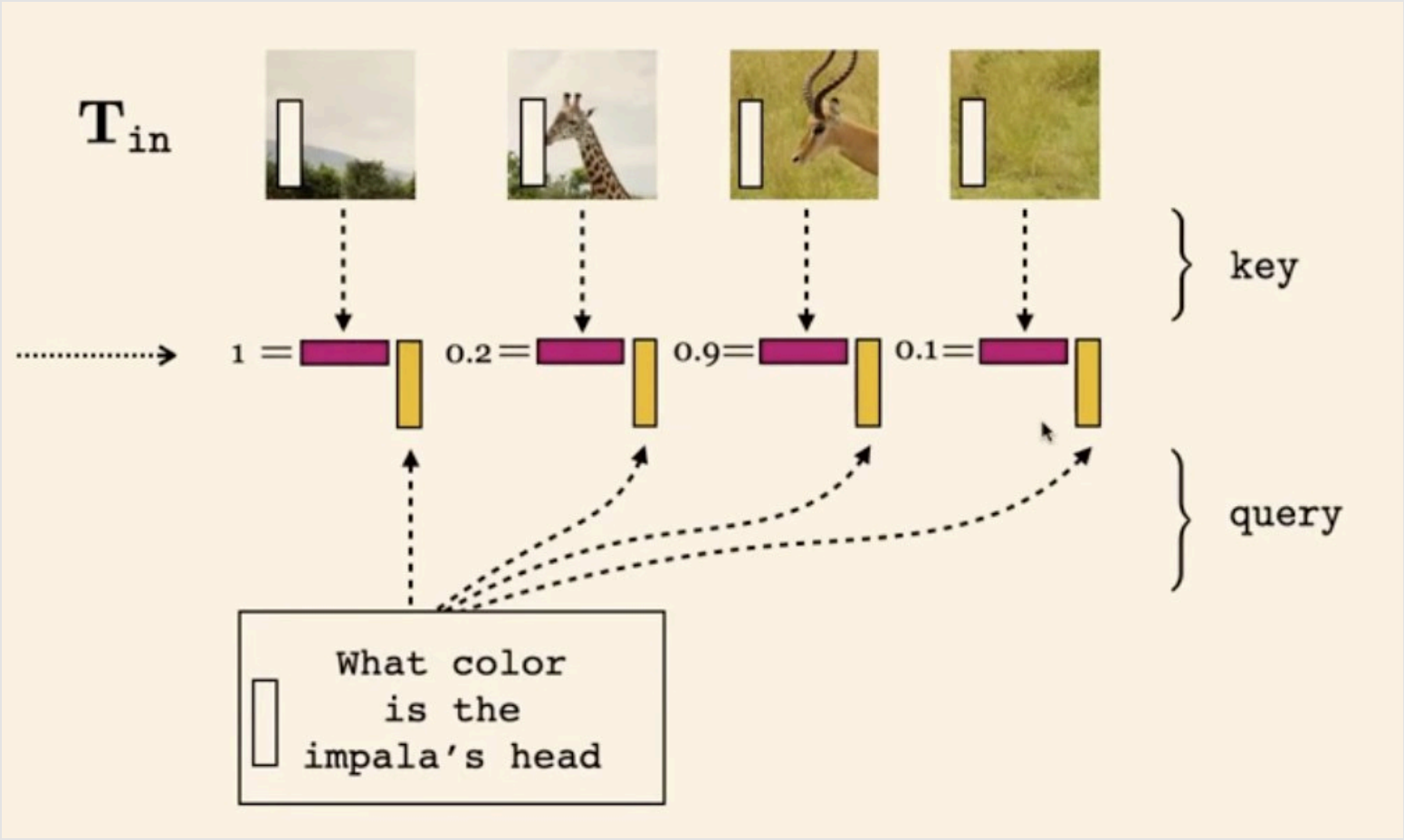
- attention에서는 더이상 free parameters가 아님
- A = function of some input data
  data → which tokens to attend to

$$A = f(\ldots)$$
$$T_{out} = AT_{in}$$

- 직관적으로
  ex1) How many animals are in the photo?
  attend → animals의 head.
  ex2) What is the color of the impala?
  attend → impala
- 이런 식으로 중요한 정보에 attend해야 한다.

query-key-value attention

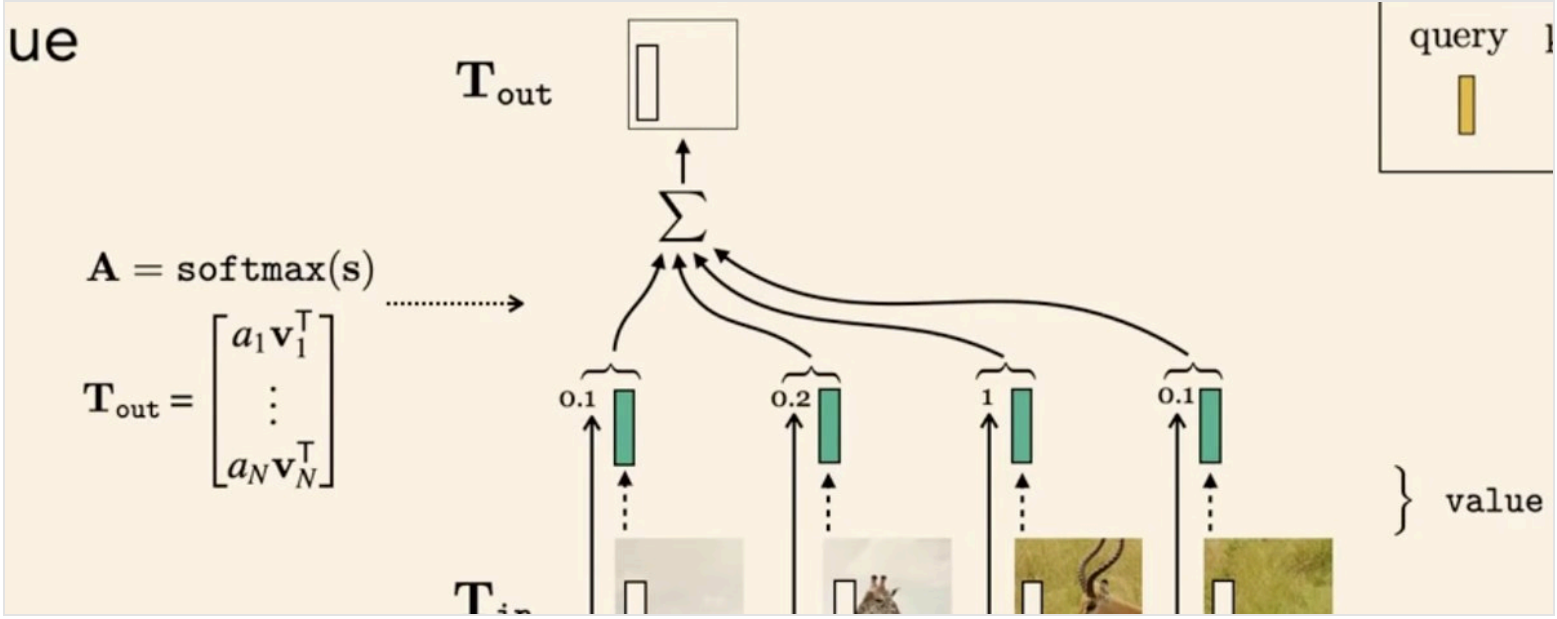
- question → attention을 판단함.
- Query : $q = W_q t$
  Key : $k = W_k t$



- (이미지라면) 각 patch들이 key를 가지고 있고,
  이곳에다가 query를 곱함으로서(유사성을 확인해봄)
  → 이게 attend할만한 것인지를 판단하는 거임

$$\mathbf{s} = [\mathbf{q}_{\text{question}}^T \mathbf{k}_1, \ldots, \mathbf{q}_{\text{question}}^T \mathbf{k}_N]$$

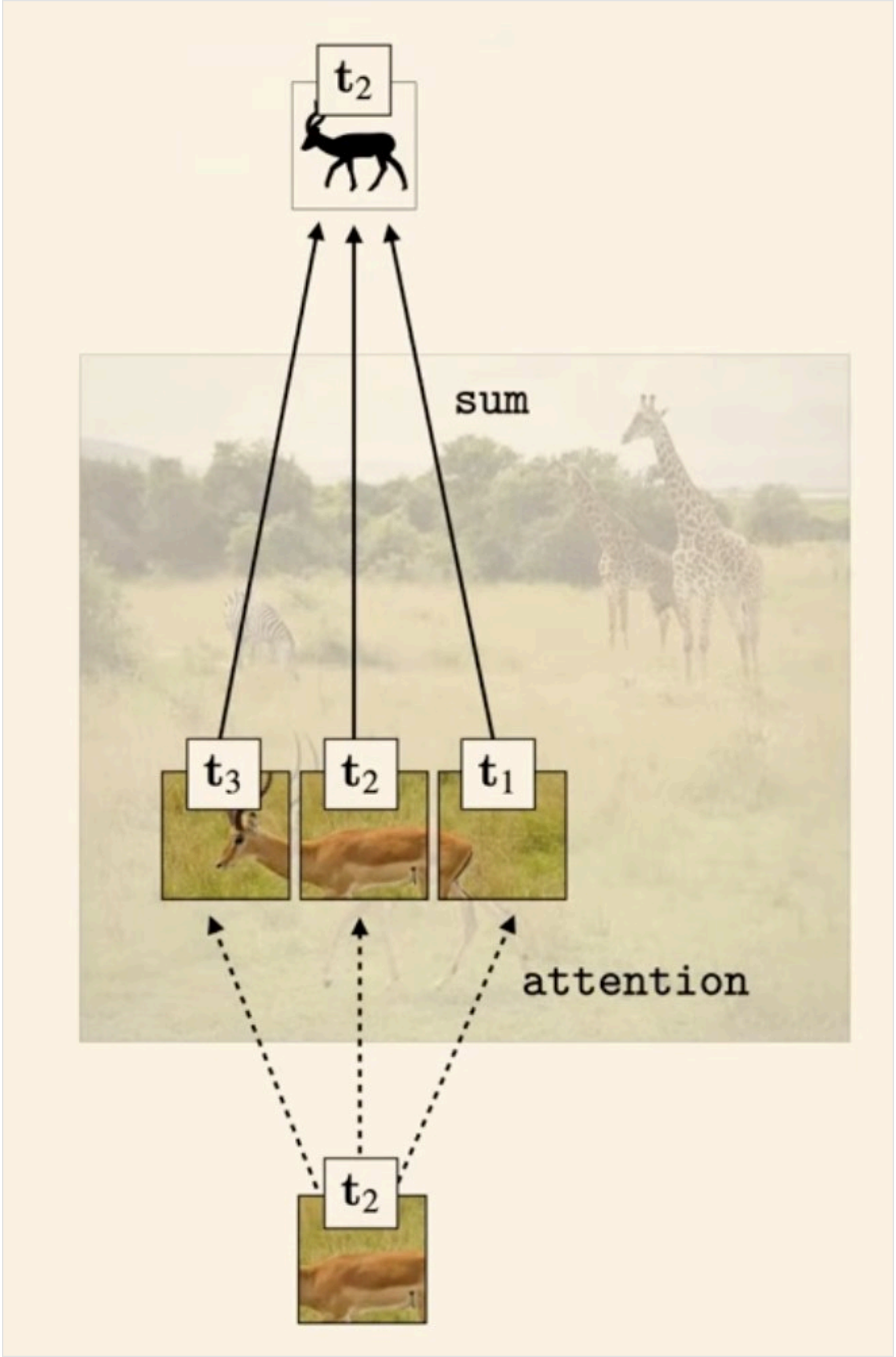
- 이런 내적값(s : 유사율)이 됨.
- Value : $v = W_v t$



- A = softmax(s) ← attention(query·질문과의 유사율로 볼 수 있음)
- value · A(attention) → T_{out}

---

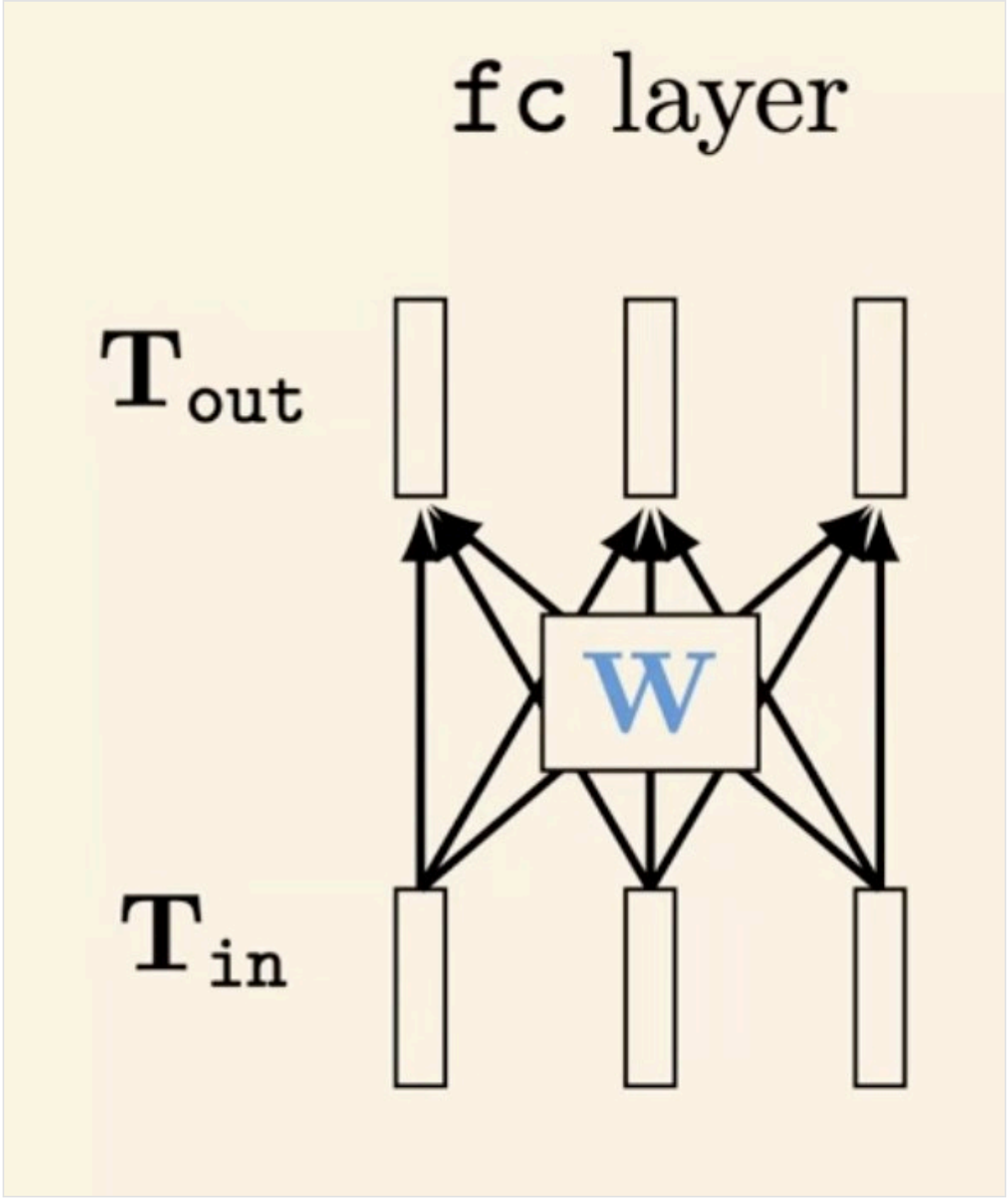
self attention

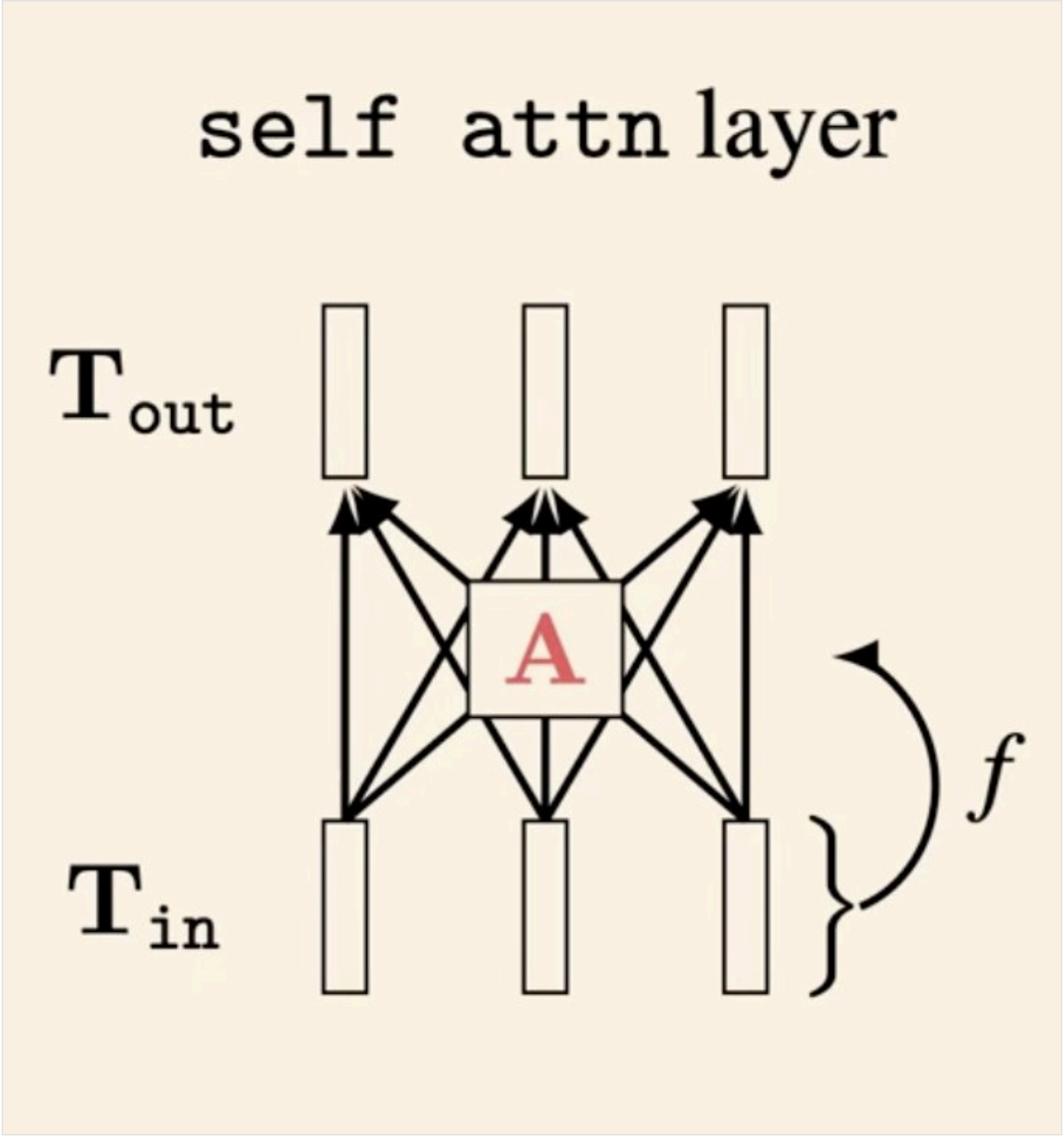

t2
sum
t3
t2
t1
attention
t2

- 외부 question으로부터 Attention하는 게 아니라,
  data 자체가 스스로 attention하는 거임.
- ex) t2는 혼자 있는 것보다, t3,t1이랑 유사도가 있음.

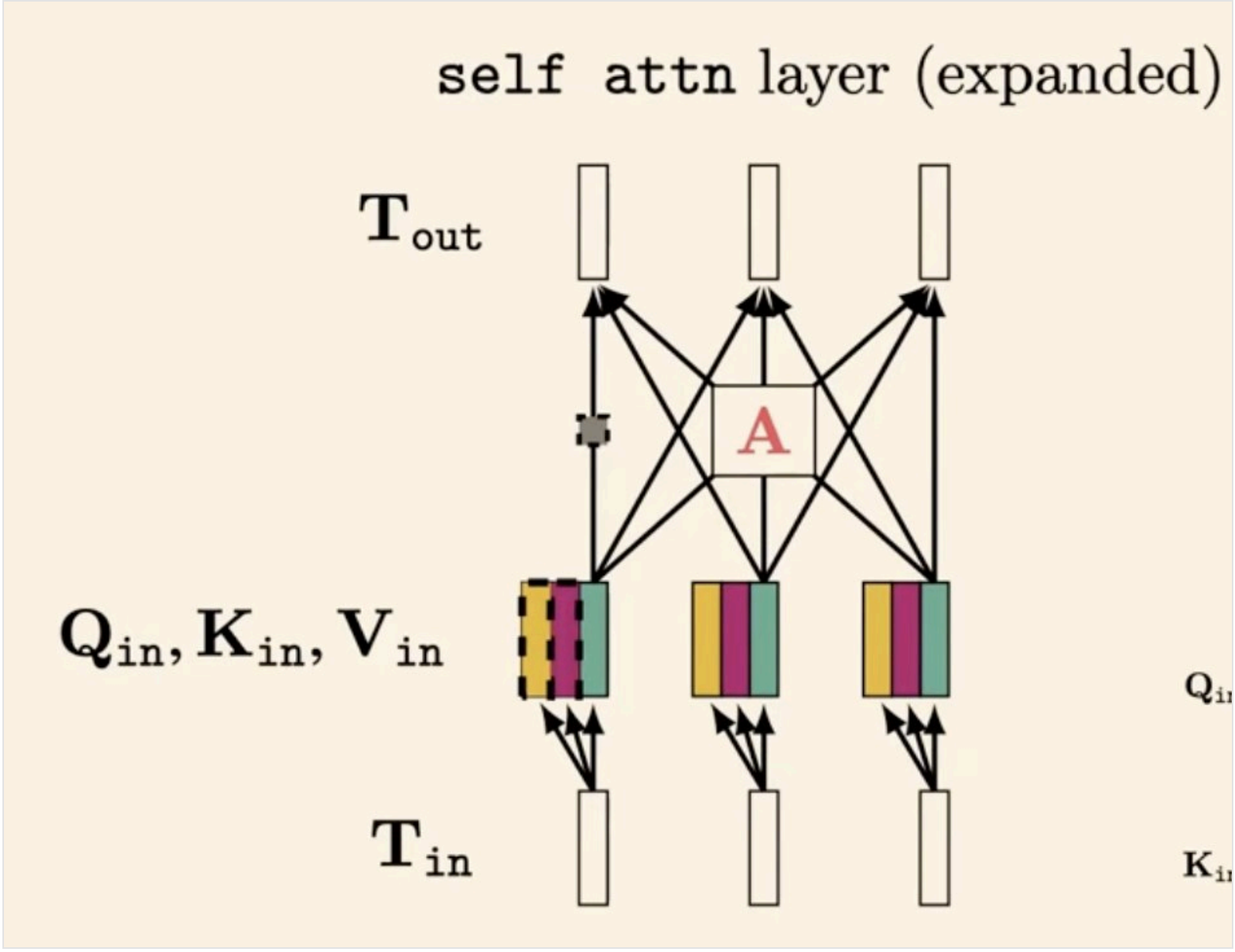


- 이 attention을 이용하면, 유사한 token끼리 segmentation할 수도 있음.


fc layer
T_{out}
W
T_{in}



Attention mathematics

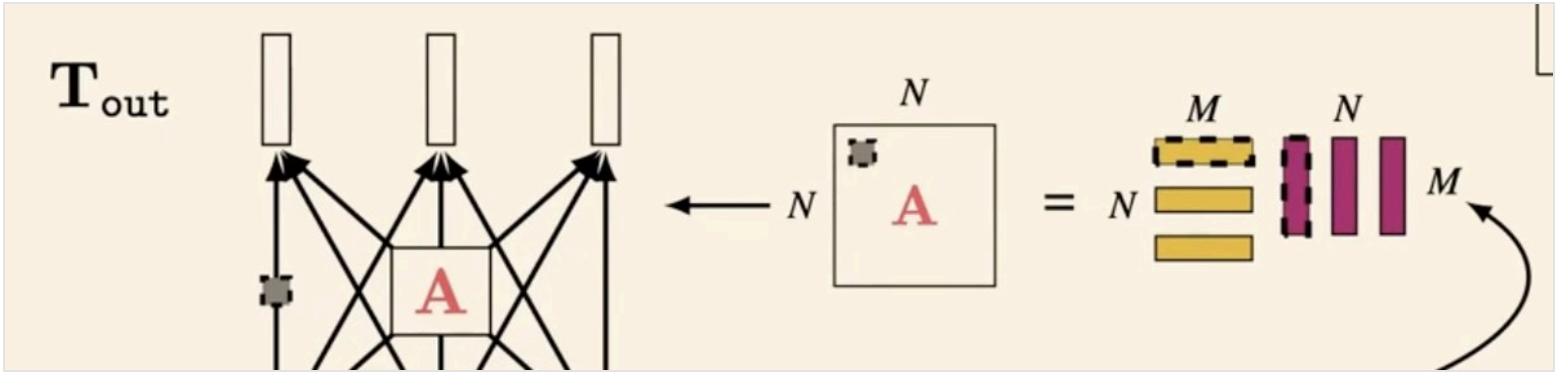

self attn layer (expanded)
$\mathbf{T}_{\text{out}}$
A
$\mathbf{Q}_{\text{in}}, \mathbf{K}_{\text{in}}, \mathbf{V}_{\text{in}}$
$\mathbf{T}_{\text{in}}$

$$\mathbf{Q}_{\text{in}} = \begin{bmatrix} \mathbf{q}_1^\top \\ \vdots \\ \mathbf{q}_N^\top \end{bmatrix} = \begin{bmatrix} (\mathbf{W}_q \mathbf{t}_1)^\top \\ \vdots \\ (\mathbf{W}_q \mathbf{t}_N)^\top \end{bmatrix} = \mathbf{T}_{\text{in}} \mathbf{W}_q^\top \quad \triangleleft \quad \text{query matrix}$$

$$\mathbf{K}_{\text{in}} = \begin{bmatrix} \mathbf{k}_1^\top \\ \vdots \\ \mathbf{k}_N^\top \end{bmatrix} = \begin{bmatrix} (\mathbf{W}_k \mathbf{t}_1)^\top \\ \vdots \\ (\mathbf{W}_k \mathbf{t}_N)^\top \end{bmatrix} = \mathbf{T}_{\text{in}} \mathbf{W}_k^\top \quad \triangleleft \quad \text{key matrix}$$

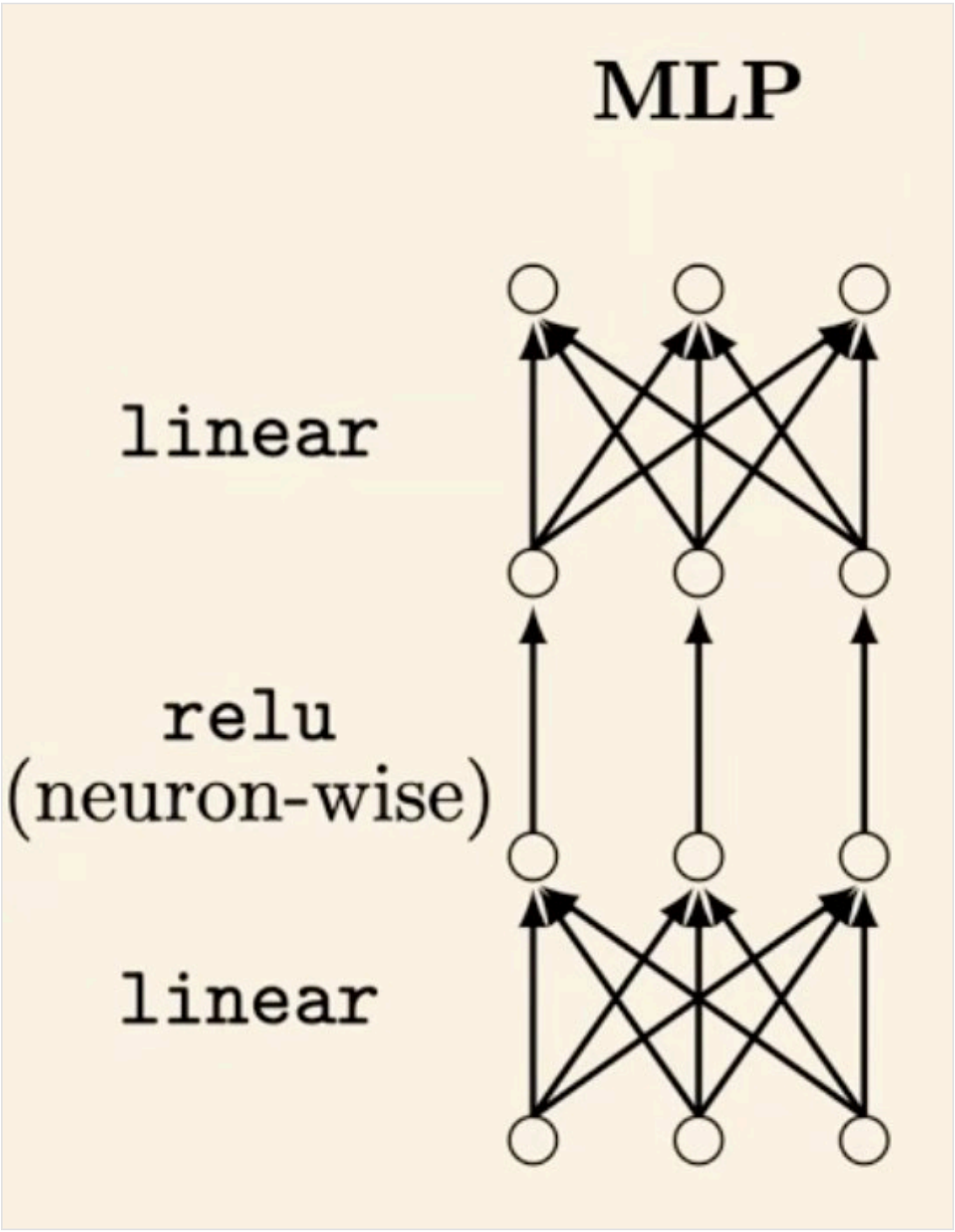
$$\mathbf{V}_{\text{in}} = \begin{bmatrix} \mathbf{v}_1^\top \\ \vdots \\ \mathbf{v}_N^\top \end{bmatrix} = \begin{bmatrix} (\mathbf{W}_v \mathbf{t}_1)^\top \\ \vdots \\ (\mathbf{W}_v \mathbf{t}_N)^\top \end{bmatrix} = \mathbf{T}_{\text{in}} \mathbf{W}_v^\top \quad \triangleleft \quad \text{value matrix}$$

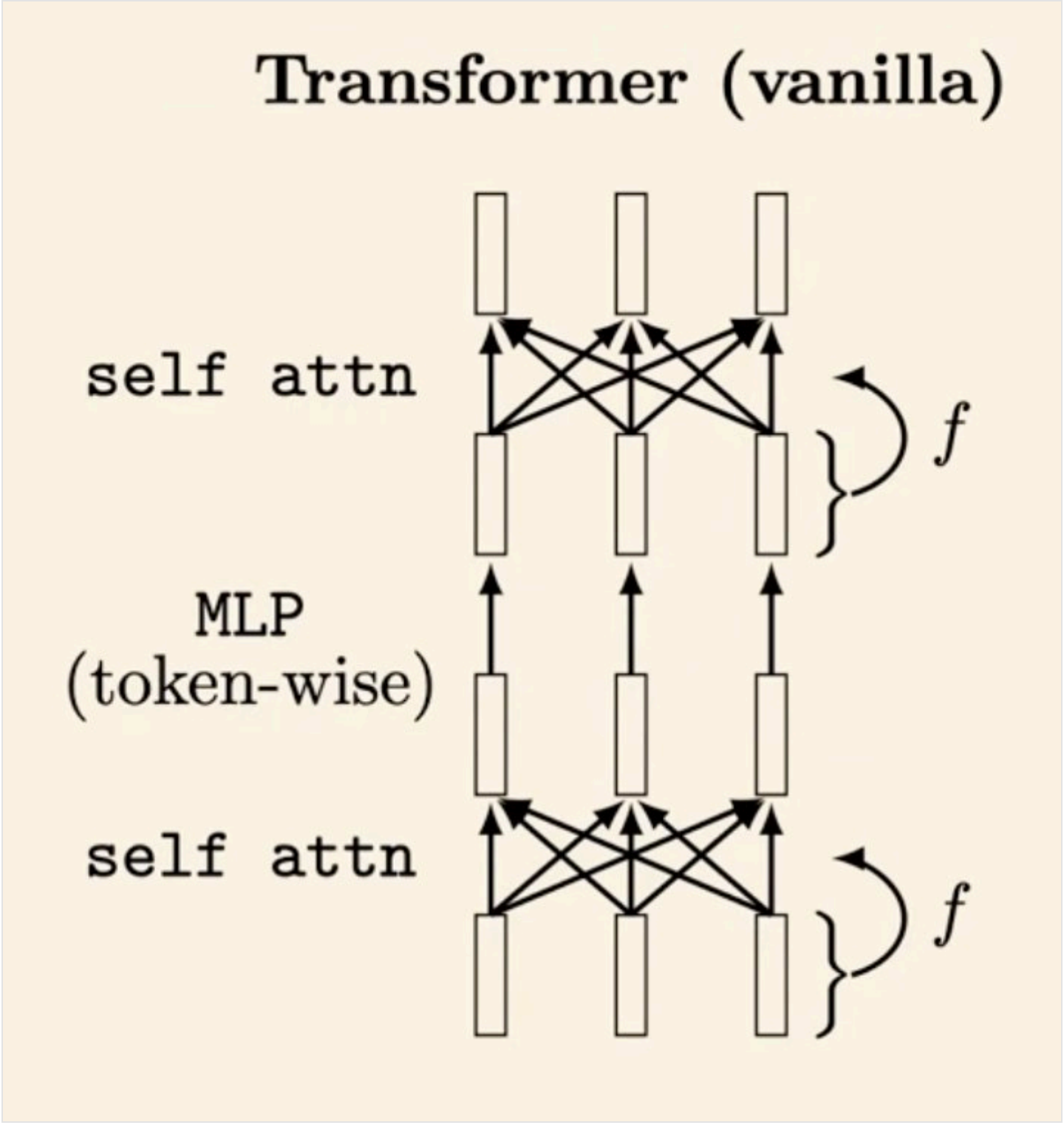
$$T_{out} = AV_{in}$$



$$\mathbf{A} = f(\mathbf{T}_{\text{in}}) = \texttt{softmax}\left(\frac{\mathbf{Q}_{\text{in}} \mathbf{K}_{\text{in}}^\top}{\sqrt{m}}\right) \quad \triangleleft \quad \text{attention matrix}$$

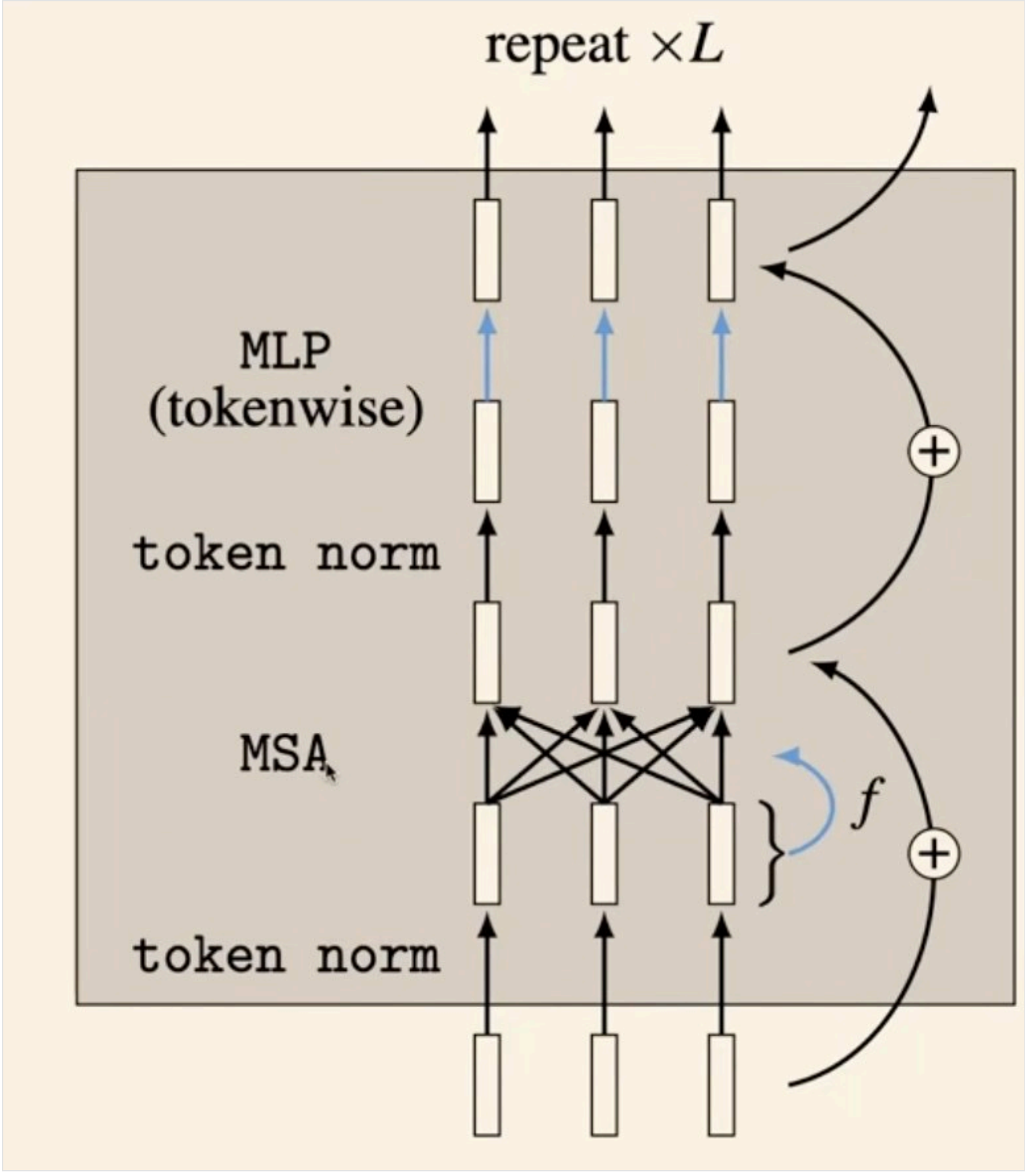
- self attention은 스스로 query와 key를 곱함 → NXN attention을 구함.


MLP
linear
relu
(neuron-wise)
linear



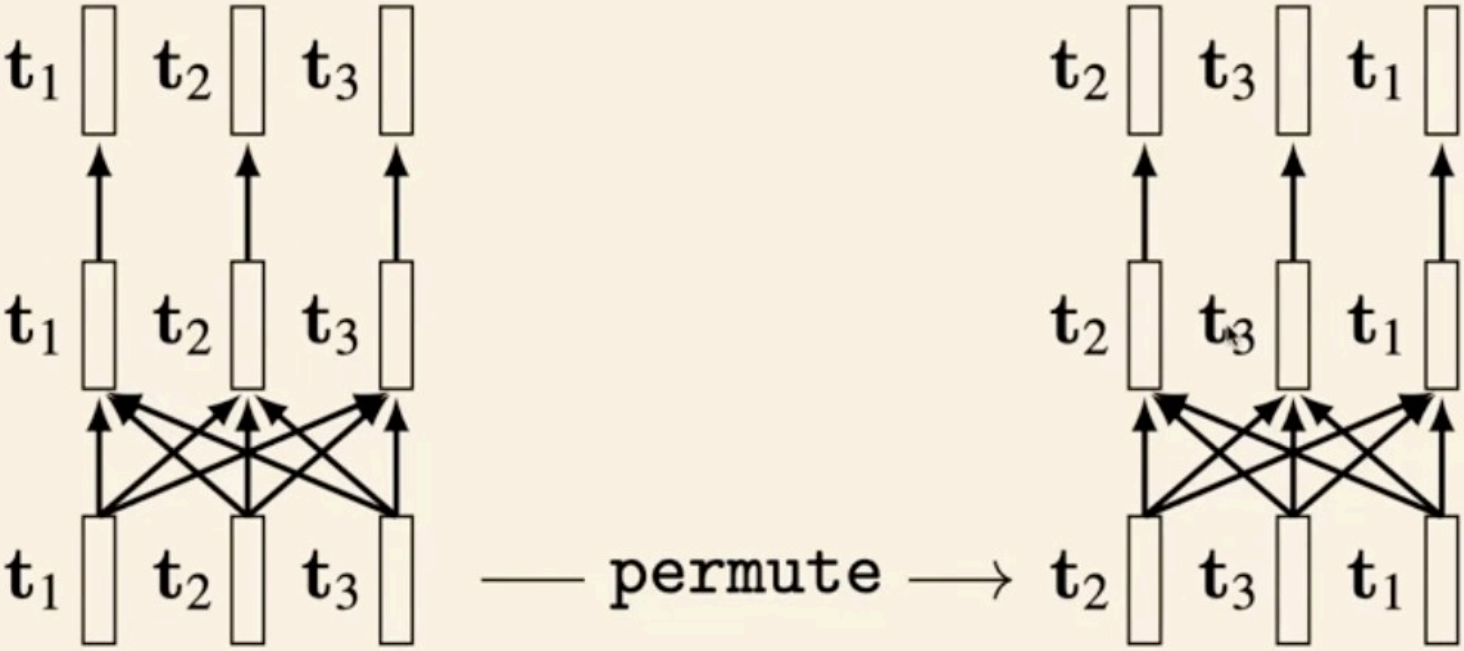
---

vision transformer(ViT)



---

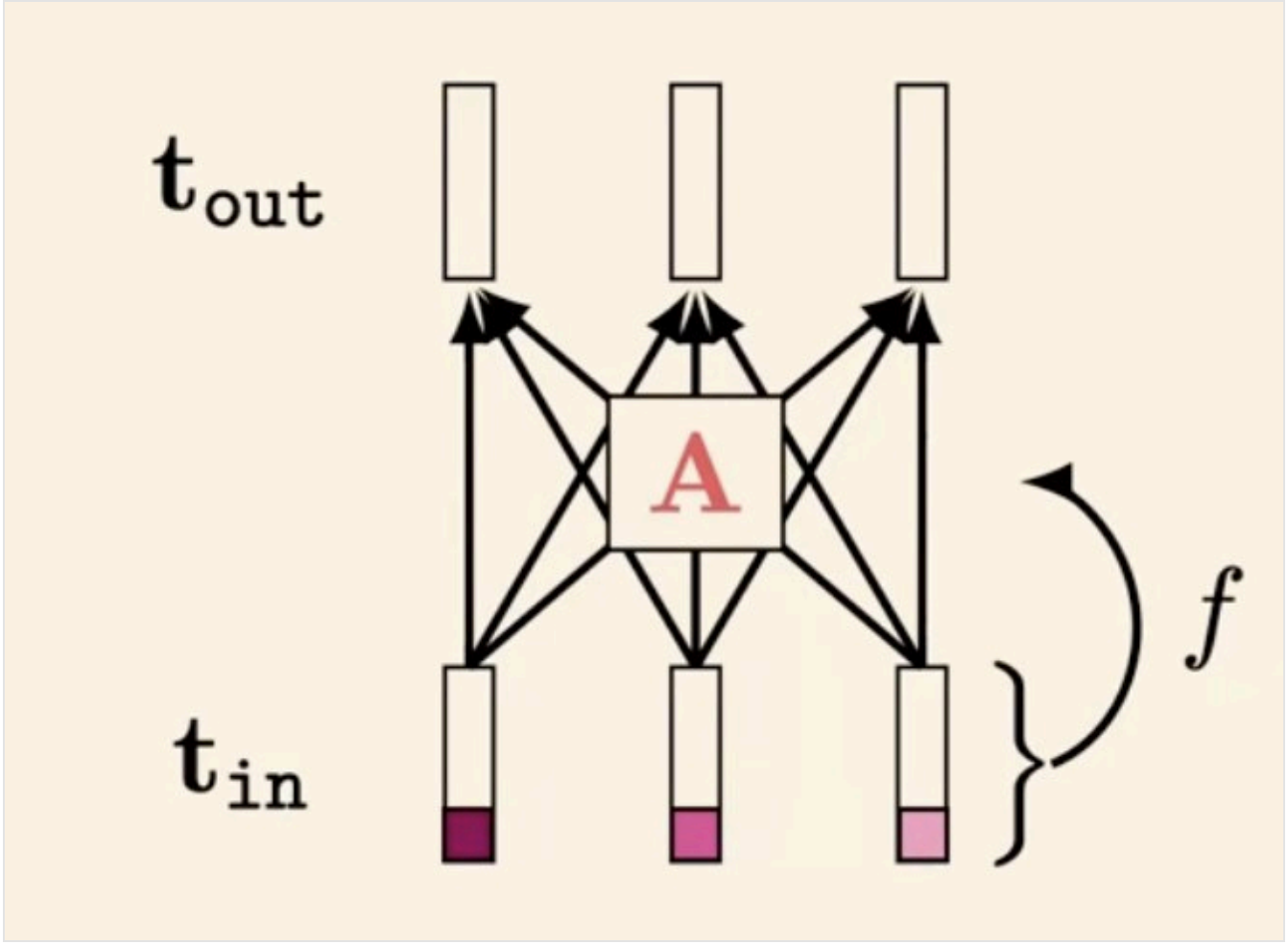
positional encoding



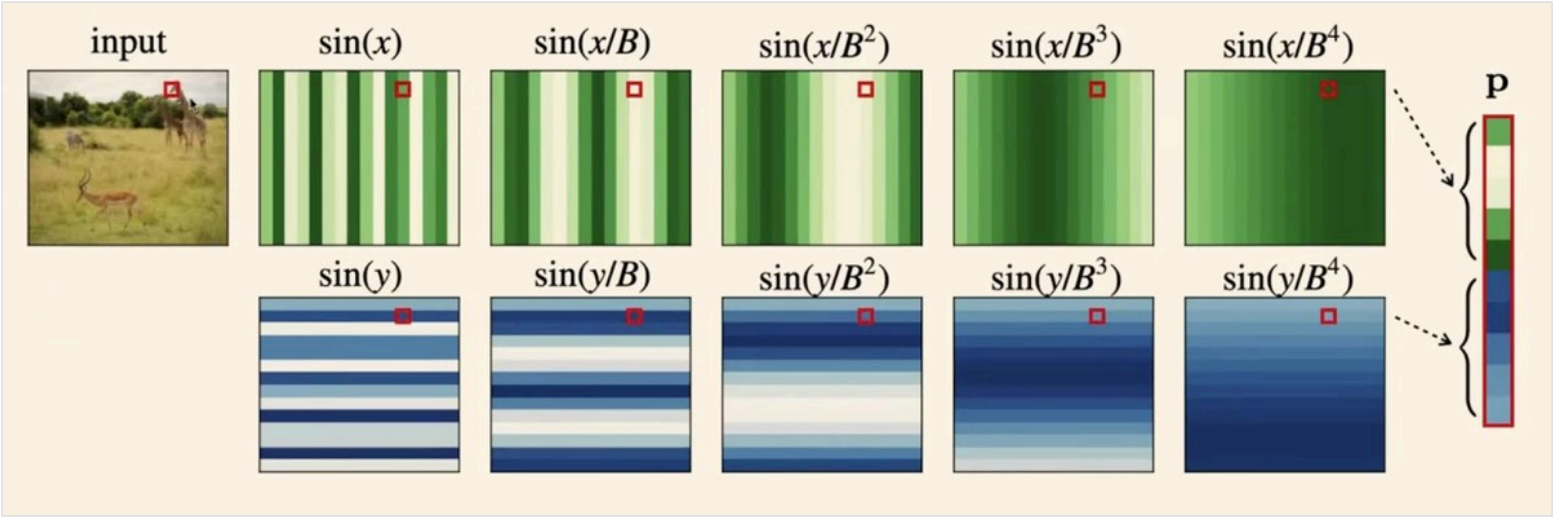
$$F_\theta(\texttt{permute}(\mathbf{T}_{\texttt{in}})) = \texttt{permute}(F_\theta(\mathbf{T}_{\texttt{in}}))$$

$$\texttt{attn}(\texttt{permute}(\mathbf{T}_{\texttt{in}})) = \texttt{permute}(\texttt{attn}(\mathbf{T}_{\texttt{in}}))$$

- token 순서를 바꿈
- position encoding은 token이 어느 곳에 있는지(이미지에서 위치/단어상 위치) 나타내는 것.



- token code vectors에다가, location information을 넣을 수 있음
  = **positional encoding**

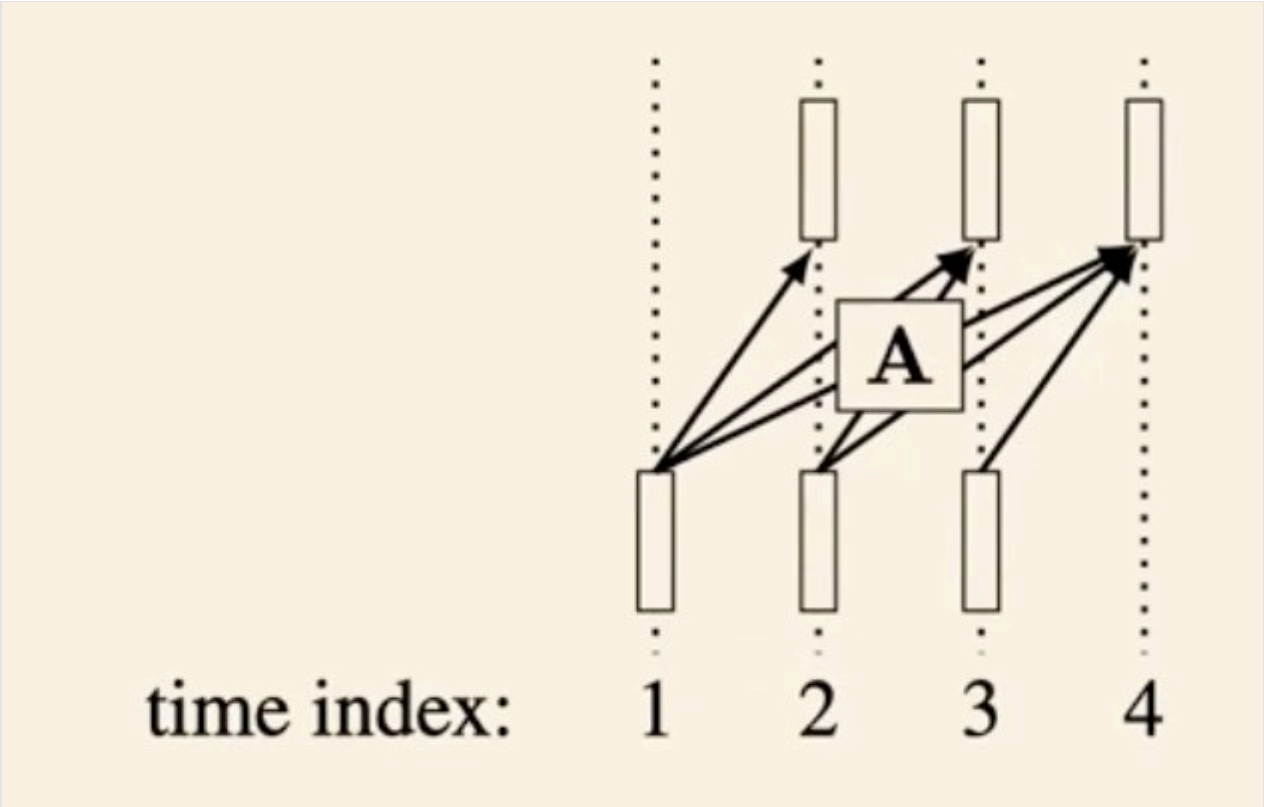


- 등등 여러 **positional encoding이 있음.**

Autoregressive models

LLM(Before)

- Once upon a time _____ → Predictor → _____
  Once ____ a time → Predictor → ____
  training set → Learner → Predictor possible word
- LLM(GPT)



- 이런 식으로 time index가 생김
- → prev token이 다음 time에 영향을 줌.

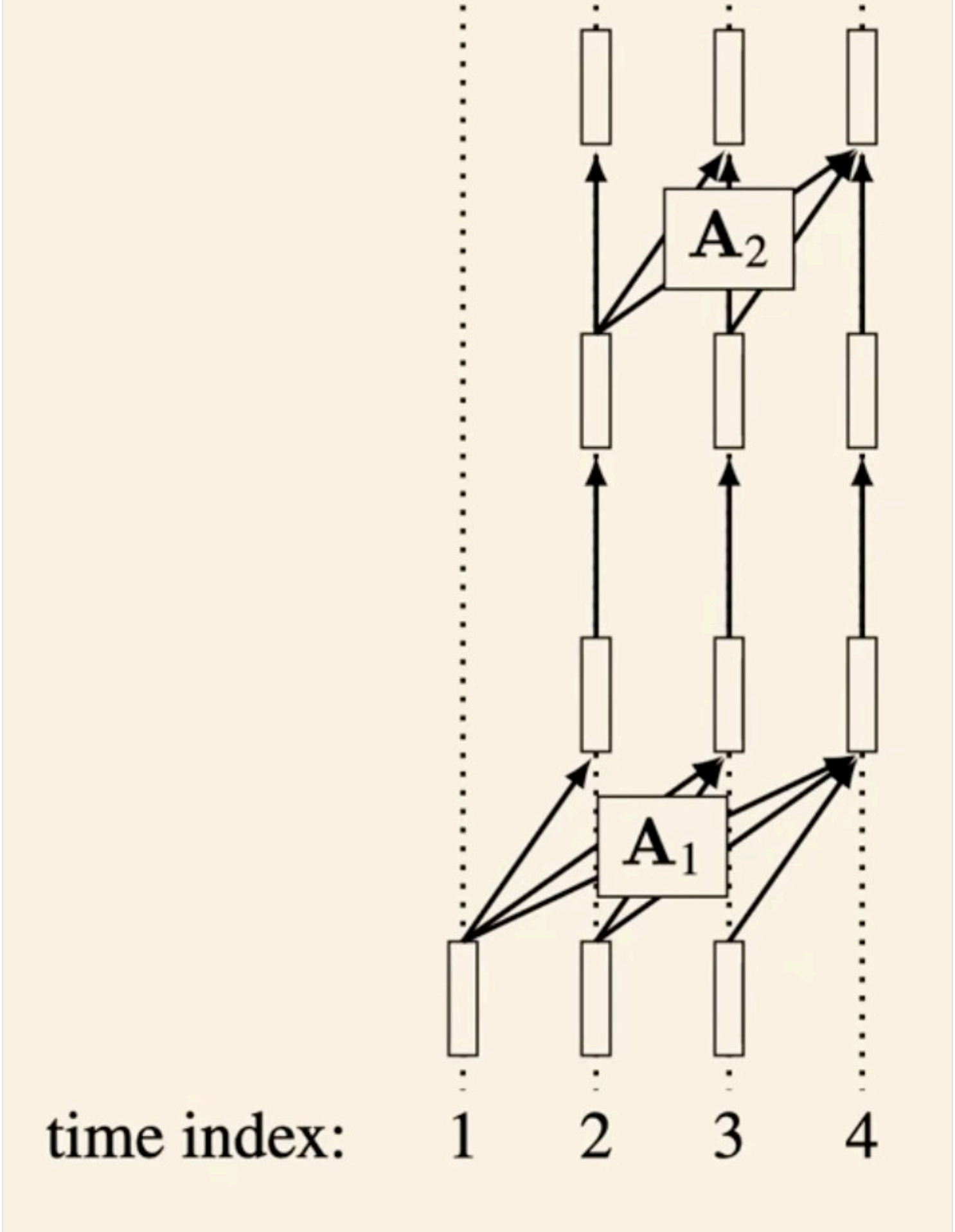
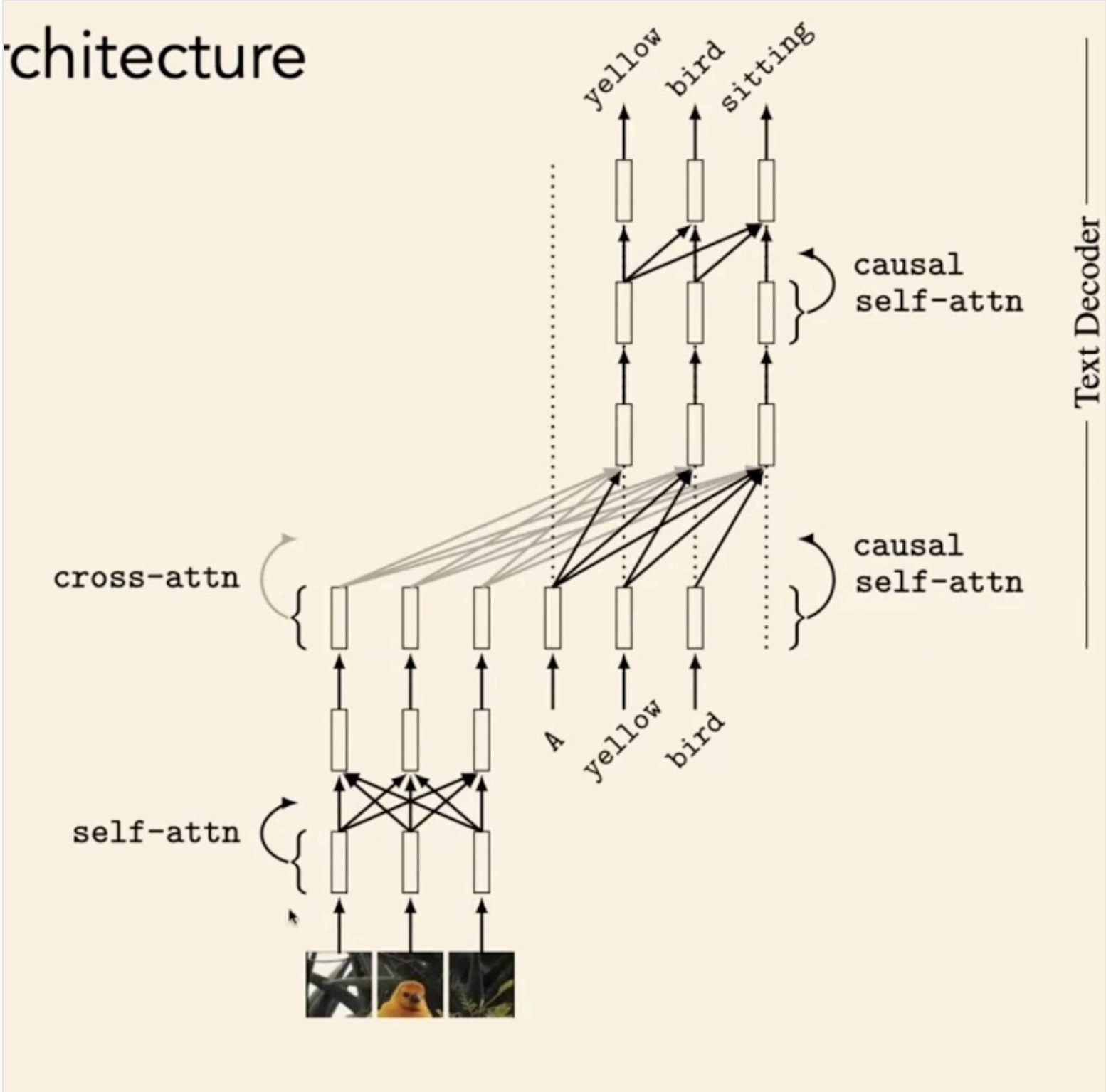
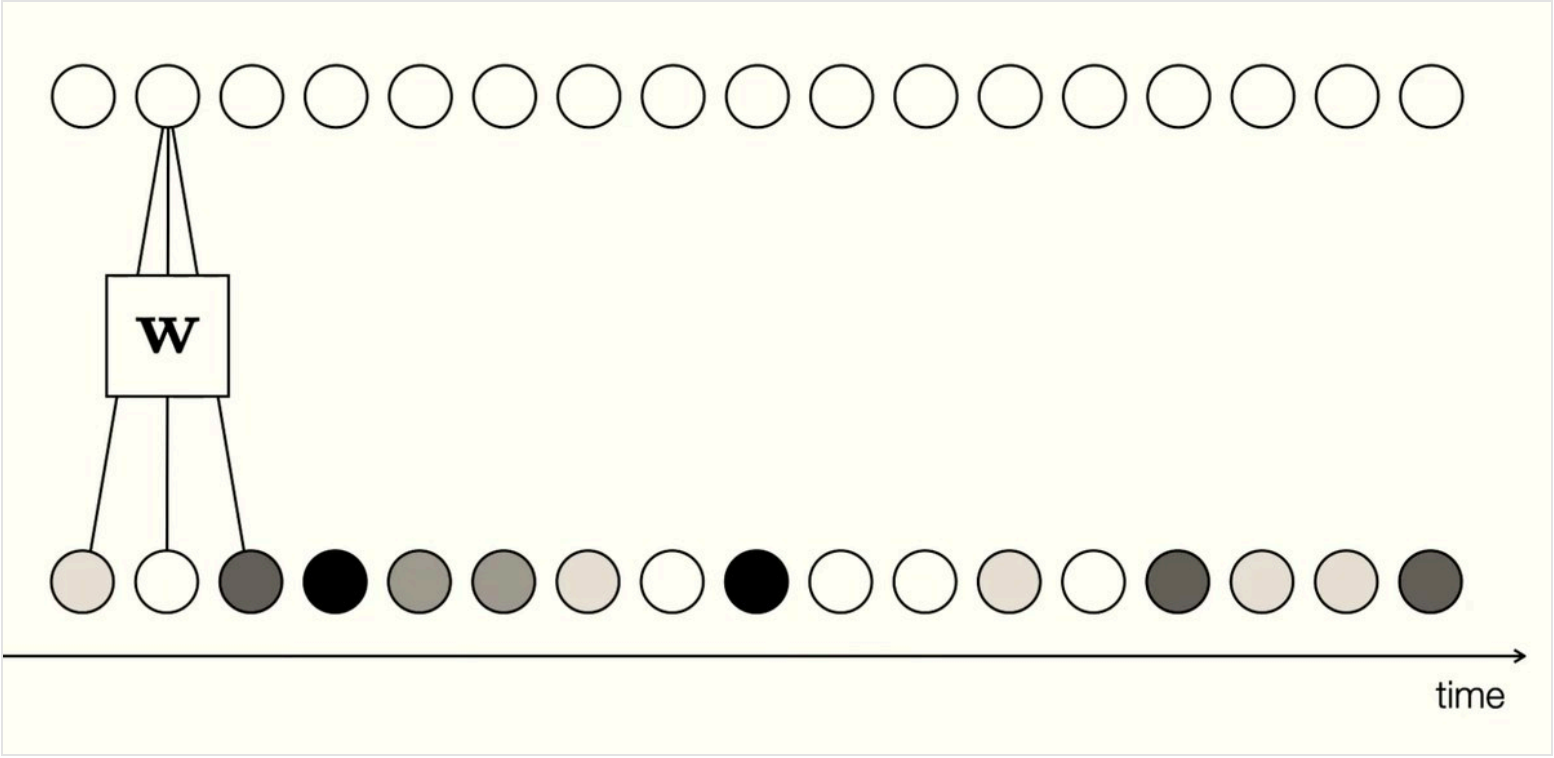


Image-to-text architecture

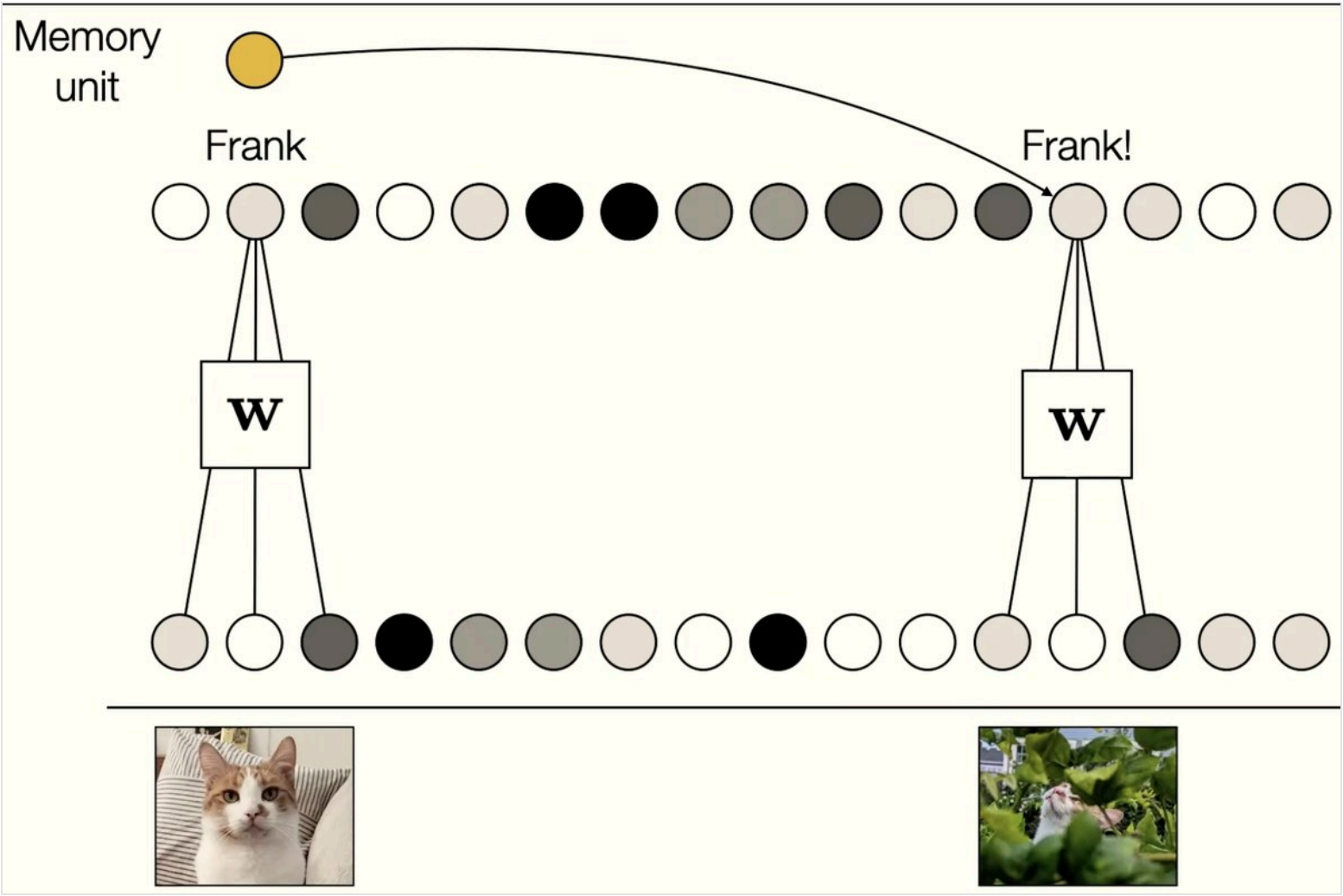


Lec. Memory

- 순차적인 것(time 등)을 모델링 하고 싶음.



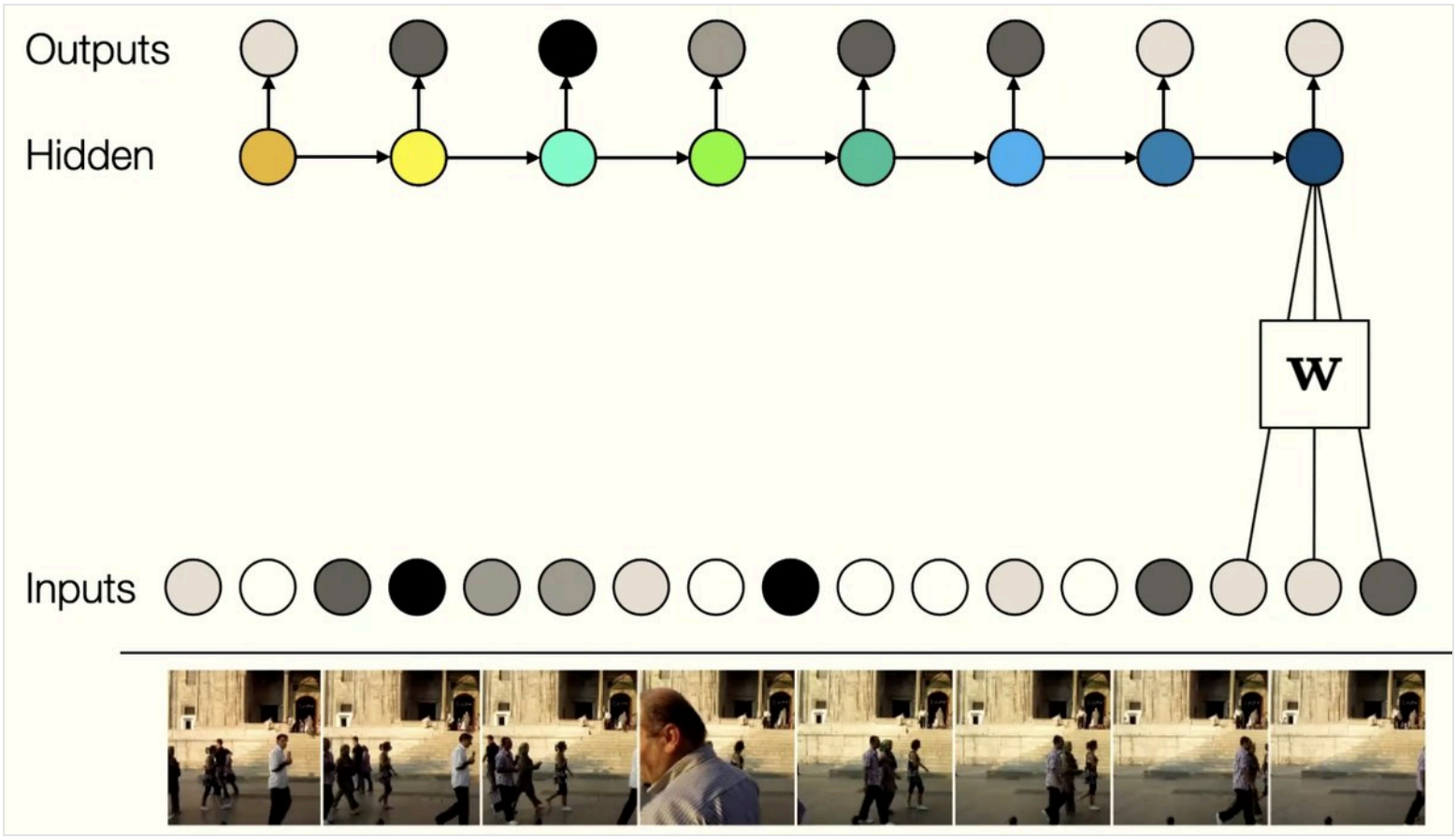
- 시간에 따라서, Convolutions할 수 있음.



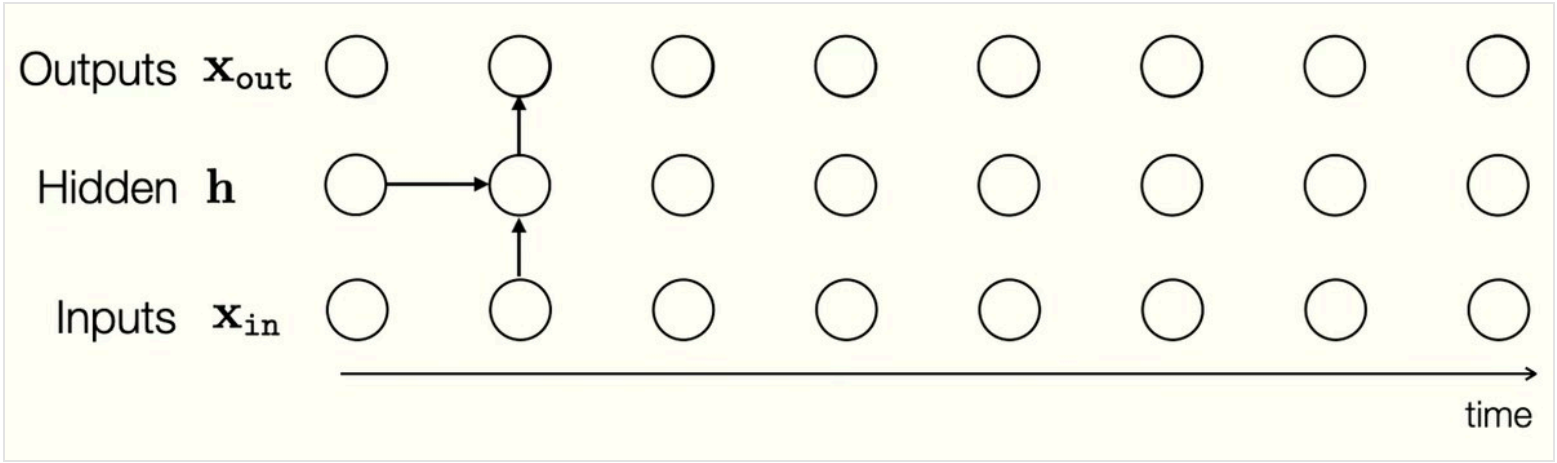
- 그런데 시간이 길어질수록 → 시간의 초반 부분을 기억하기 어려워짐.
  초반의 정보(예: 고양이)가 제대로 인식되었더라도,

후반에 이 정보를 잊고, 다시 인식하는 과정에서 다른 것으로 인식할 수 있음(예: 고양이인 걸 잊고, 호랑이라고 해버림)

- Recurrent Neural Networks(RNNs)

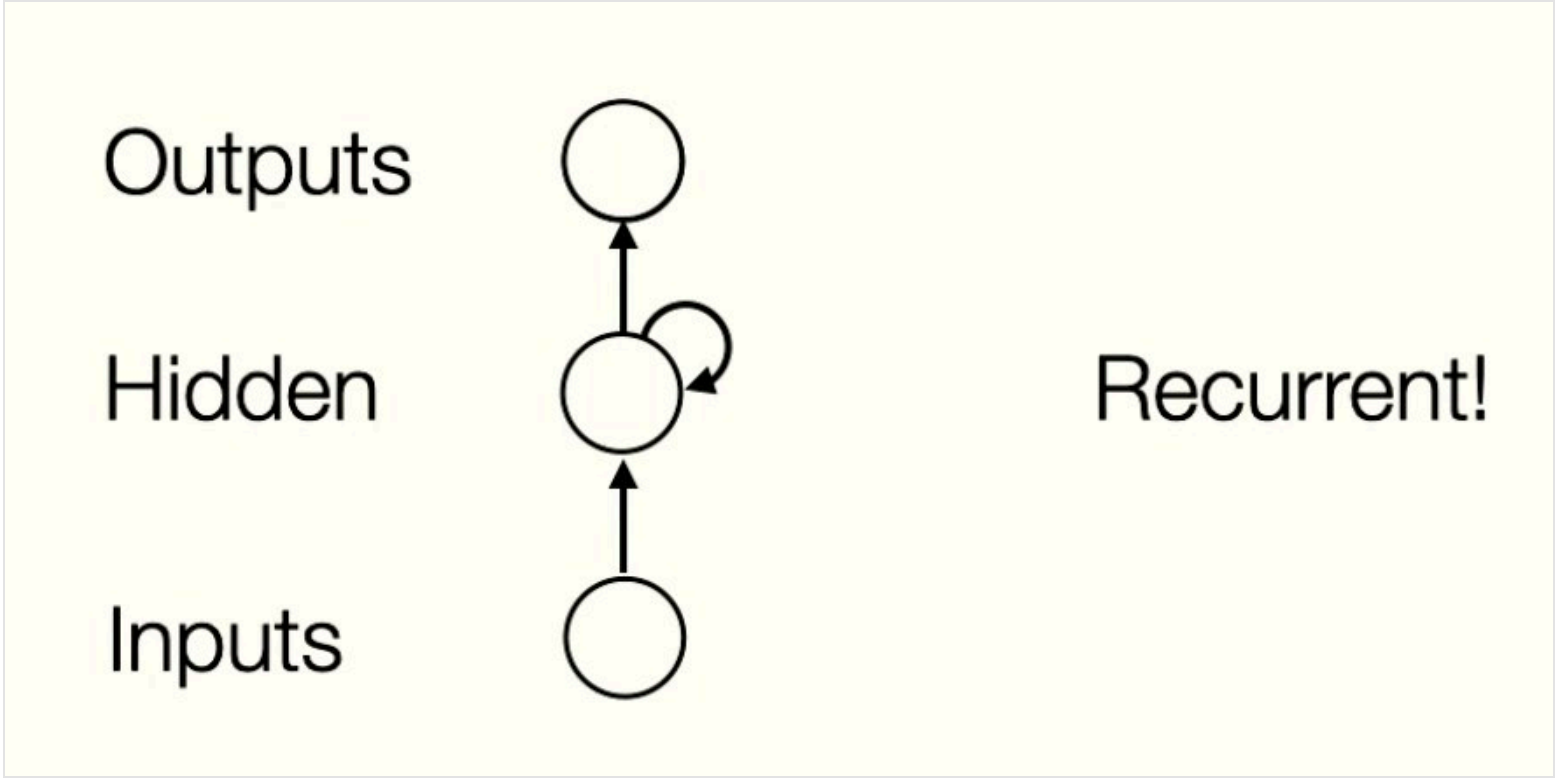


- hidden unit을 이용해서, memory까지 얻을 수 있음

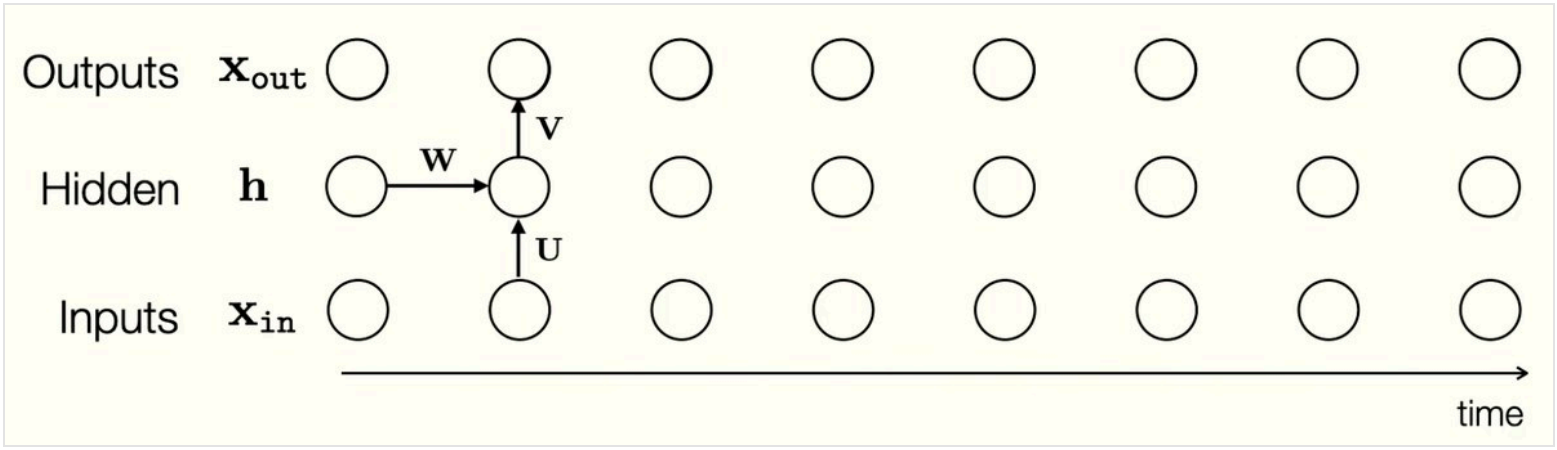


- $h_t = f(h_{t-1}, x_{in}[t])$로 표현할 수 있음. f is fix(independ on time)

$$x_{out}[t] = g(h_t)$$



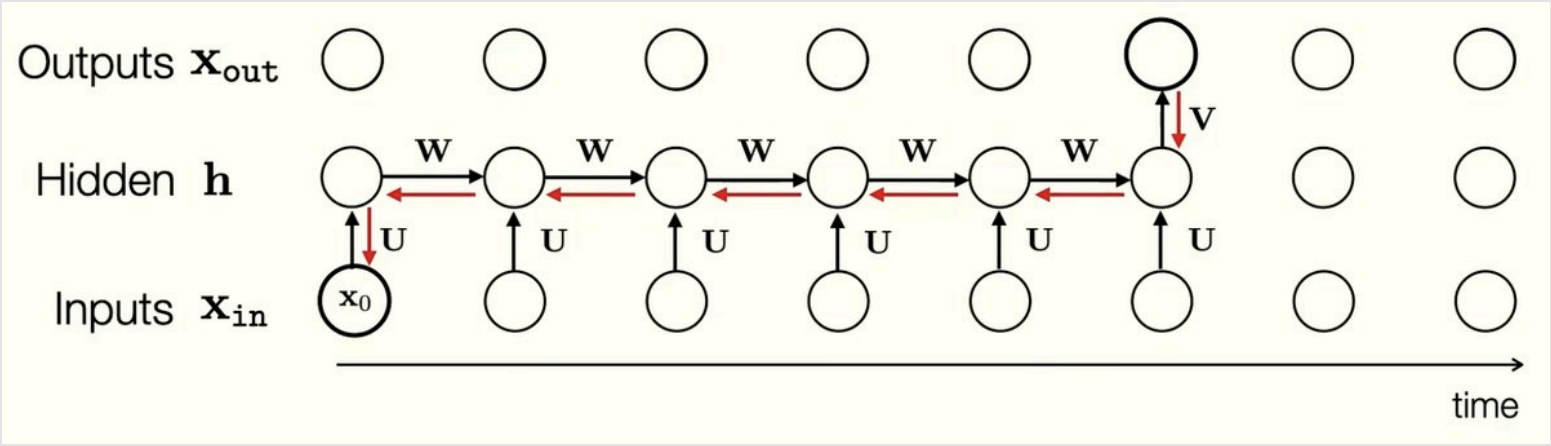
- hidden state에 loop가 있는 형태라고 볼 수 있음



- linear layer로 모두 표현할 수 있음.

$$h_t = \sigma_1(Wh_{t-1} + Ux_{in}[t] + b)$$
$$x_{out}[t] = \sigma_2(Vh_t + c)$$

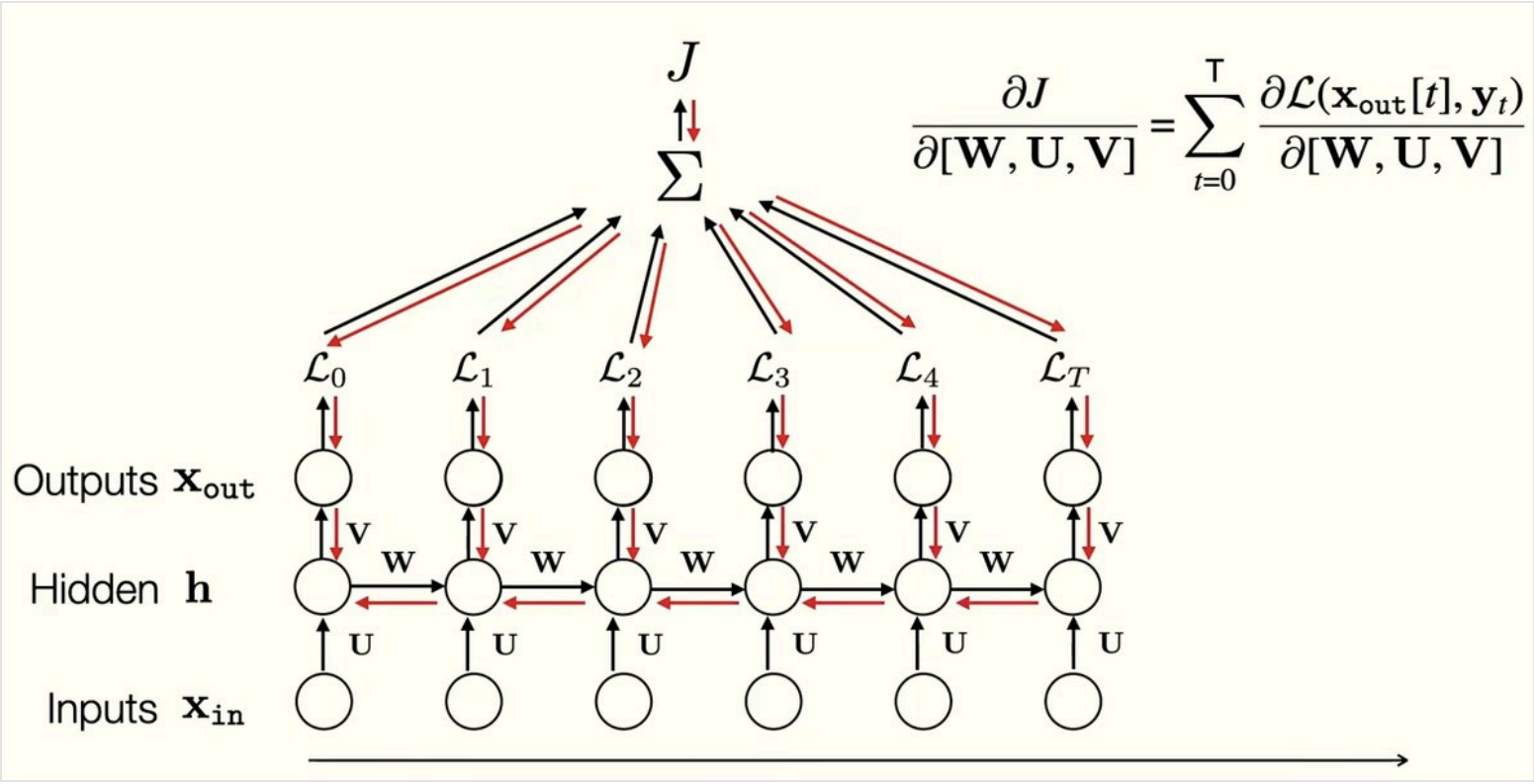
Backprop through time



- 시간에 따라 가는 그래프를 거꾸로 따라가면 됨.

$$\frac{\partial \mathbf{x}_{\text{out}}[t]}{\partial \mathbf{x}_{\text{in}}[0]} = \frac{\partial \mathbf{x}_{\text{out}}[t]}{\partial \mathbf{h}_T} \frac{\partial \mathbf{h}_T}{\partial \mathbf{h}_{T-1}} \cdots \frac{\partial \mathbf{h}_1}{\partial \mathbf{h}_0} \frac{\partial \mathbf{h}_0}{\partial \mathbf{x}_{\text{in}}[0]}$$

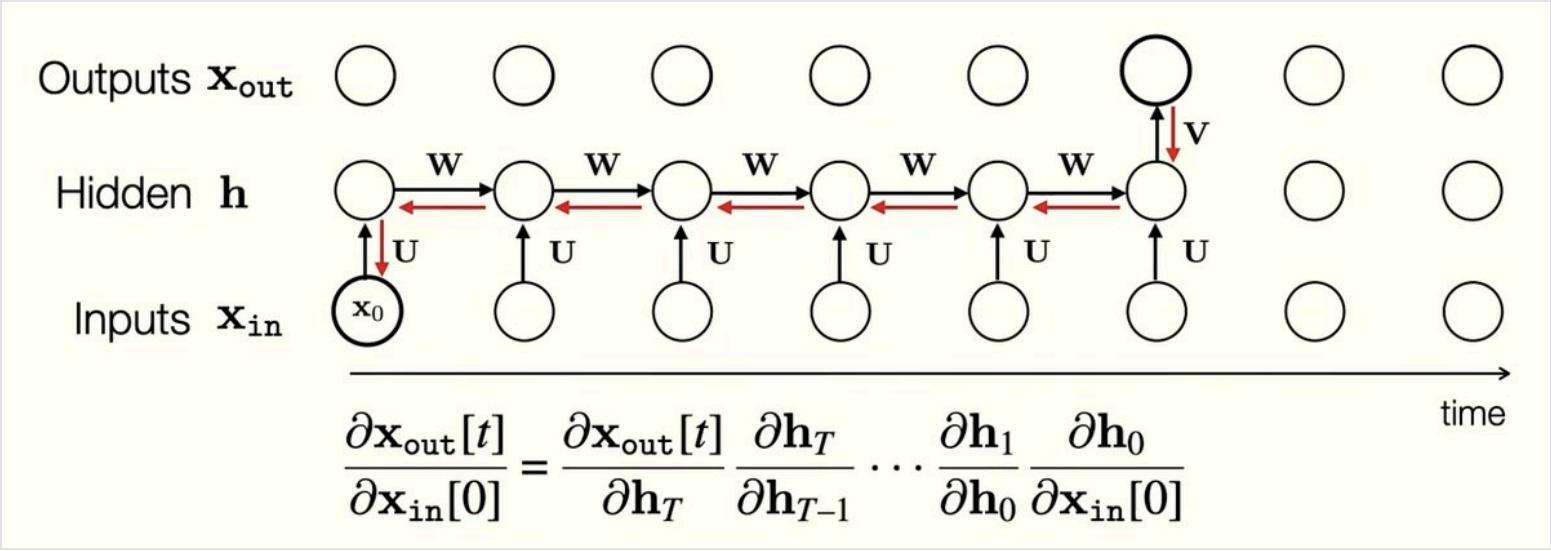
- 여기서, W가 고정되어 있음.



-

---

why not remember everything?

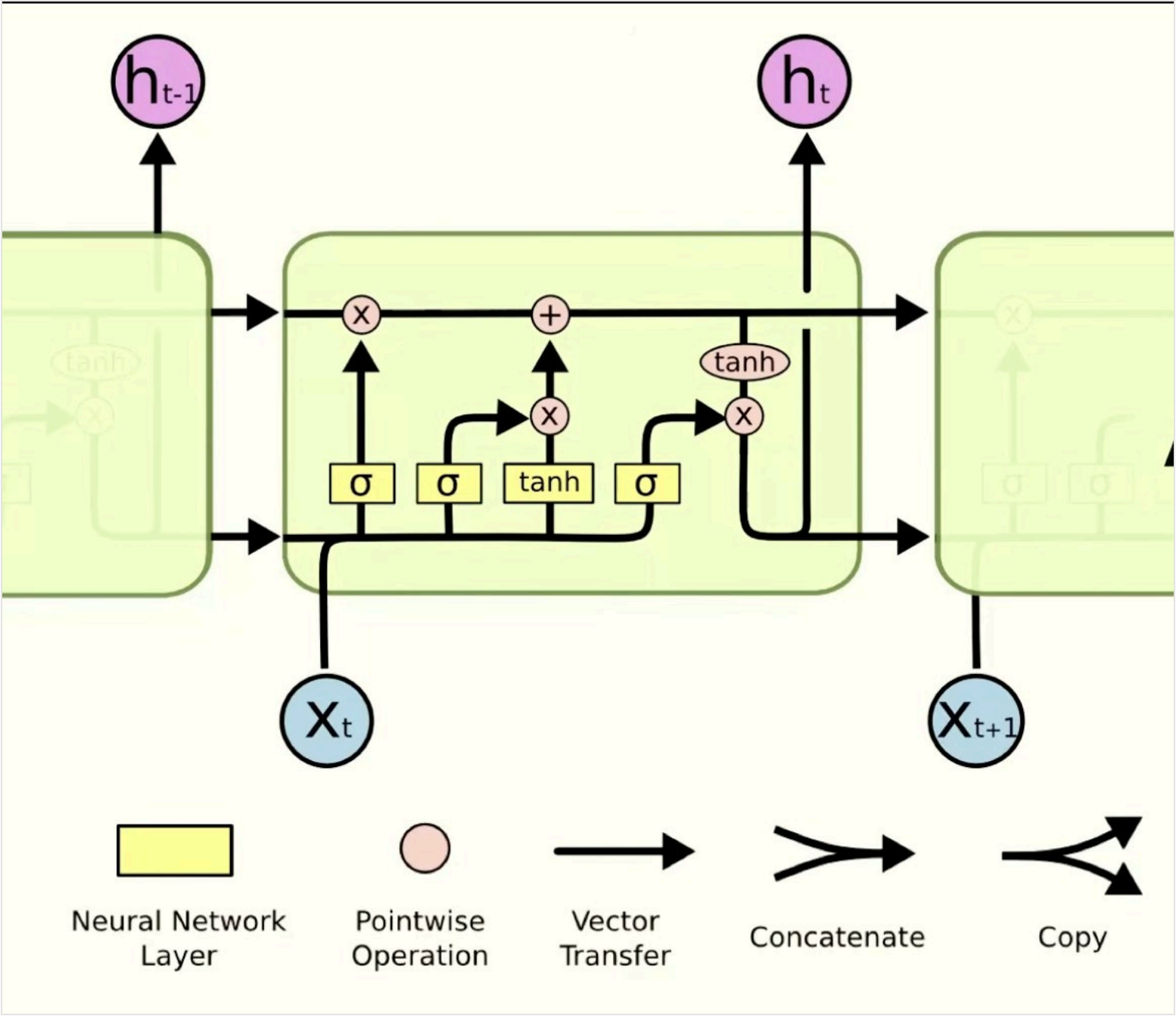


- time 차이가 큰 곳의 결과에 영향을 주려면, 많은 W를 거쳐야 함.

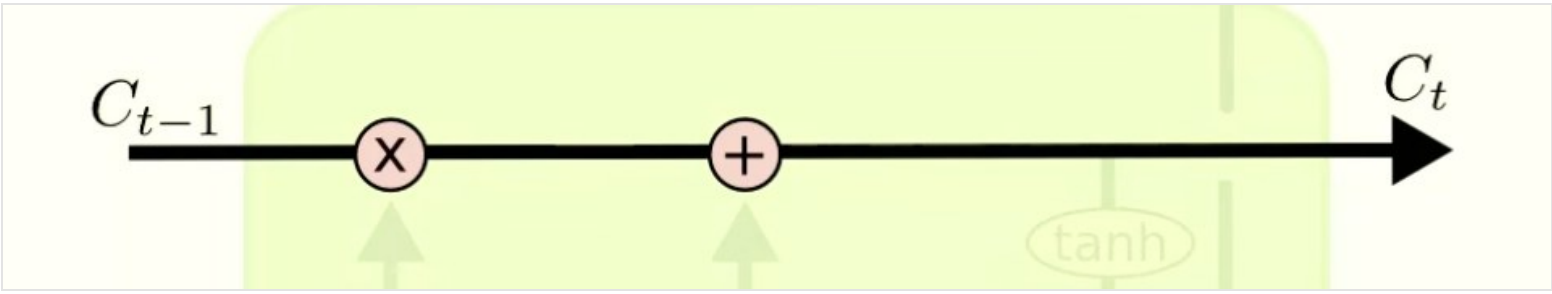
$$h_1 = Wh_0 + u_{x_1}$$
$$h_2 = W(Wh_0 + u_{x_1}) + u_{x_2}$$

- W^t 항을 가지게 됨. 이때, t가 상당히 커지는 상황에서는
  W<1이면, W항이 사라지고, W>1이면 W항이 무한대로 감.
  따라서 W가 거의 1과 가까워야 함.
- ⇒ **LSTM을 사용하기 시작함.**

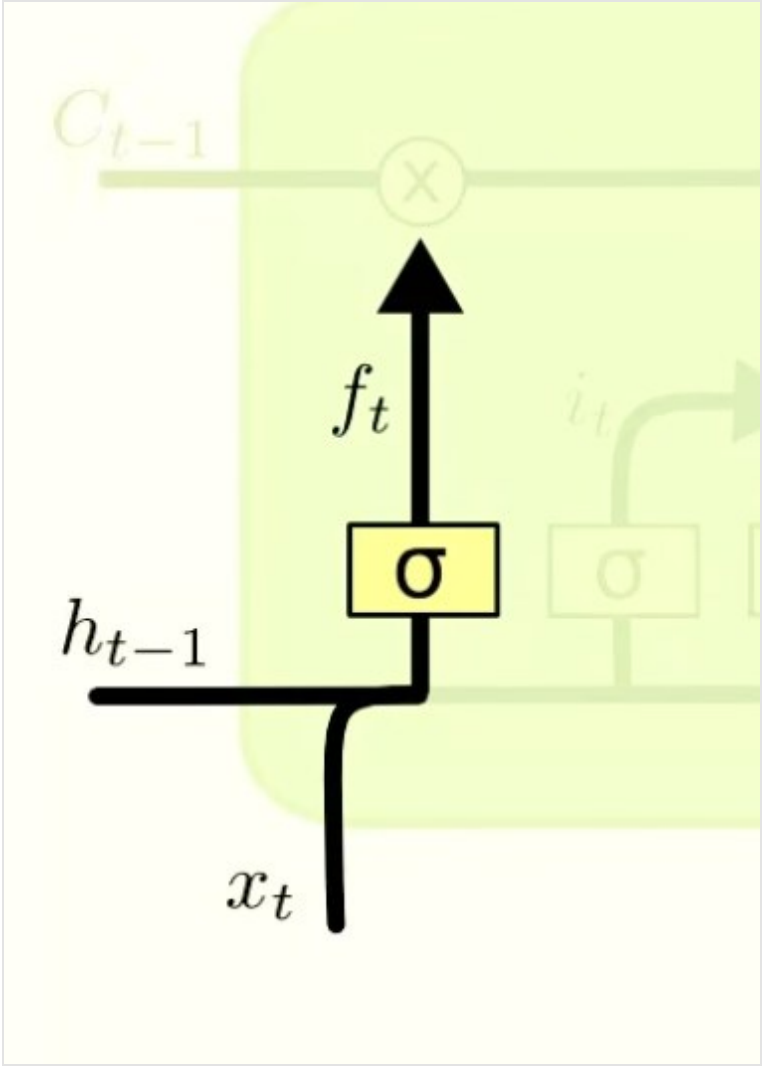
LSTM



- 저장할 것 / 잊을 것을 구분하는 장치가 있음.

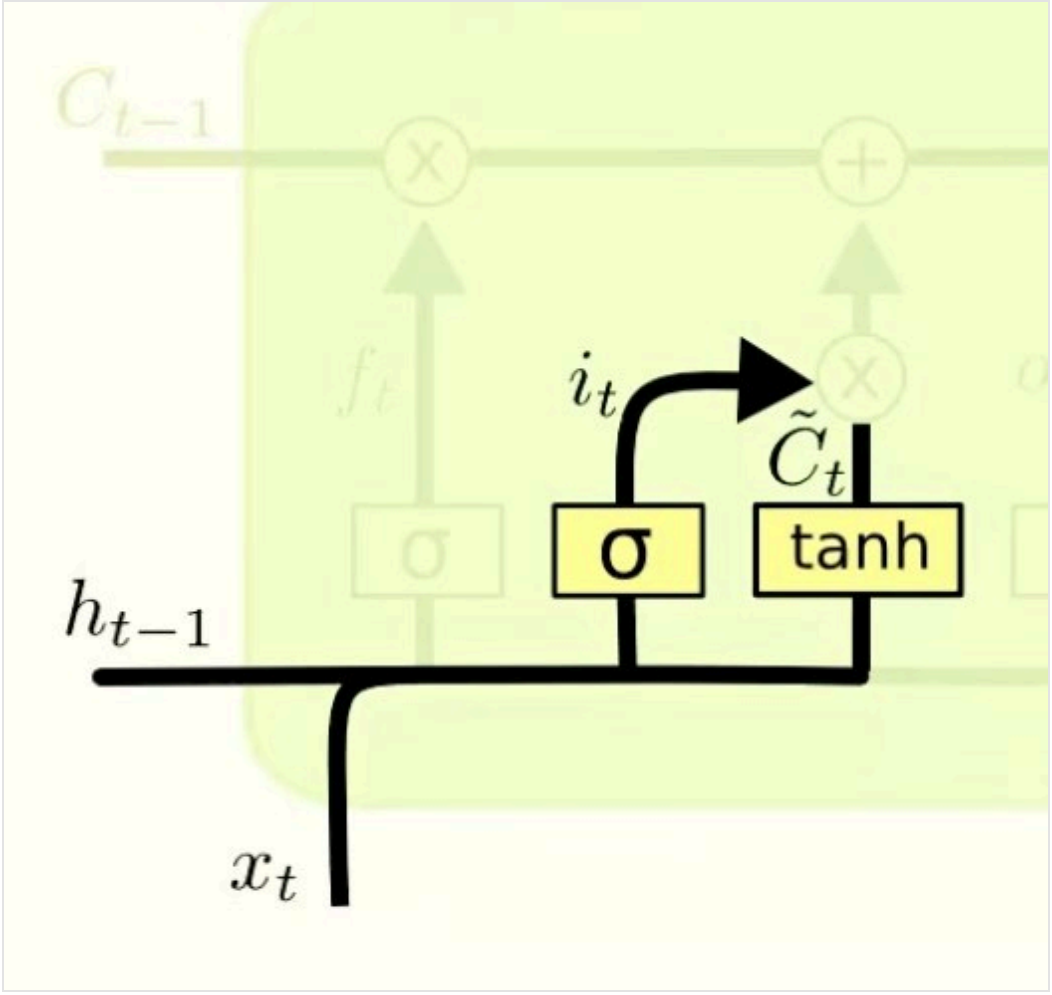


- C_t = Cell state → Long-term Memory 역할
  → 무엇을 잊고, 무엇을 추가할지 결정하는 부분에 의해 조절됨.

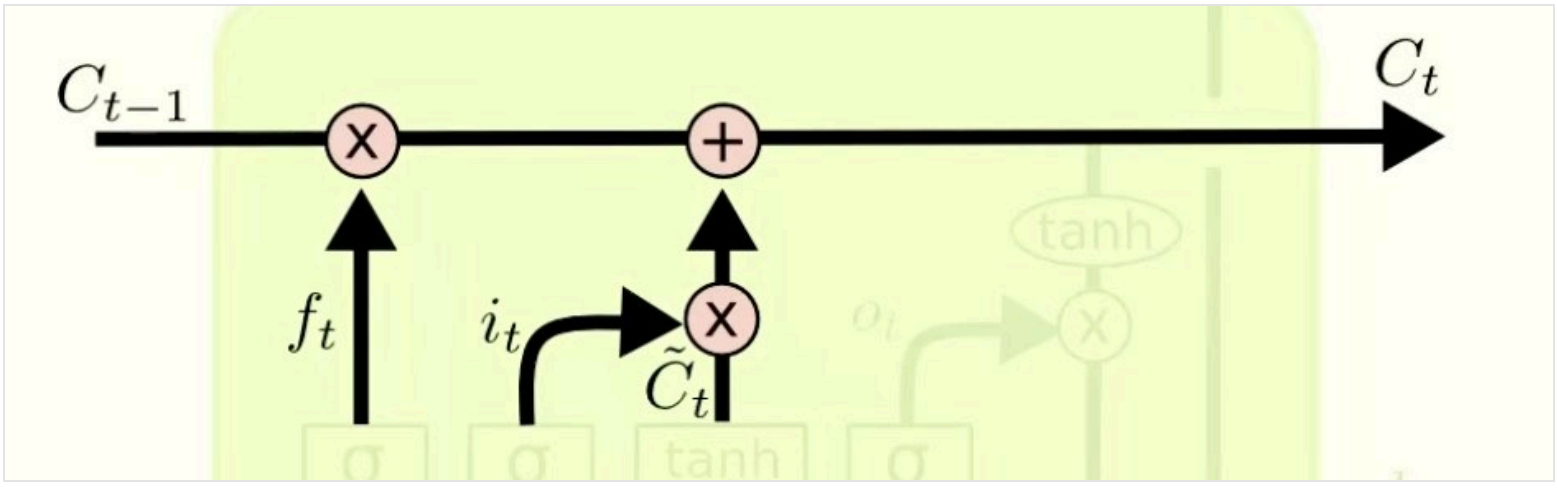


$$f_t = \sigma(W_f \cdot [h_{t-1}, x_t] + b_f)$$

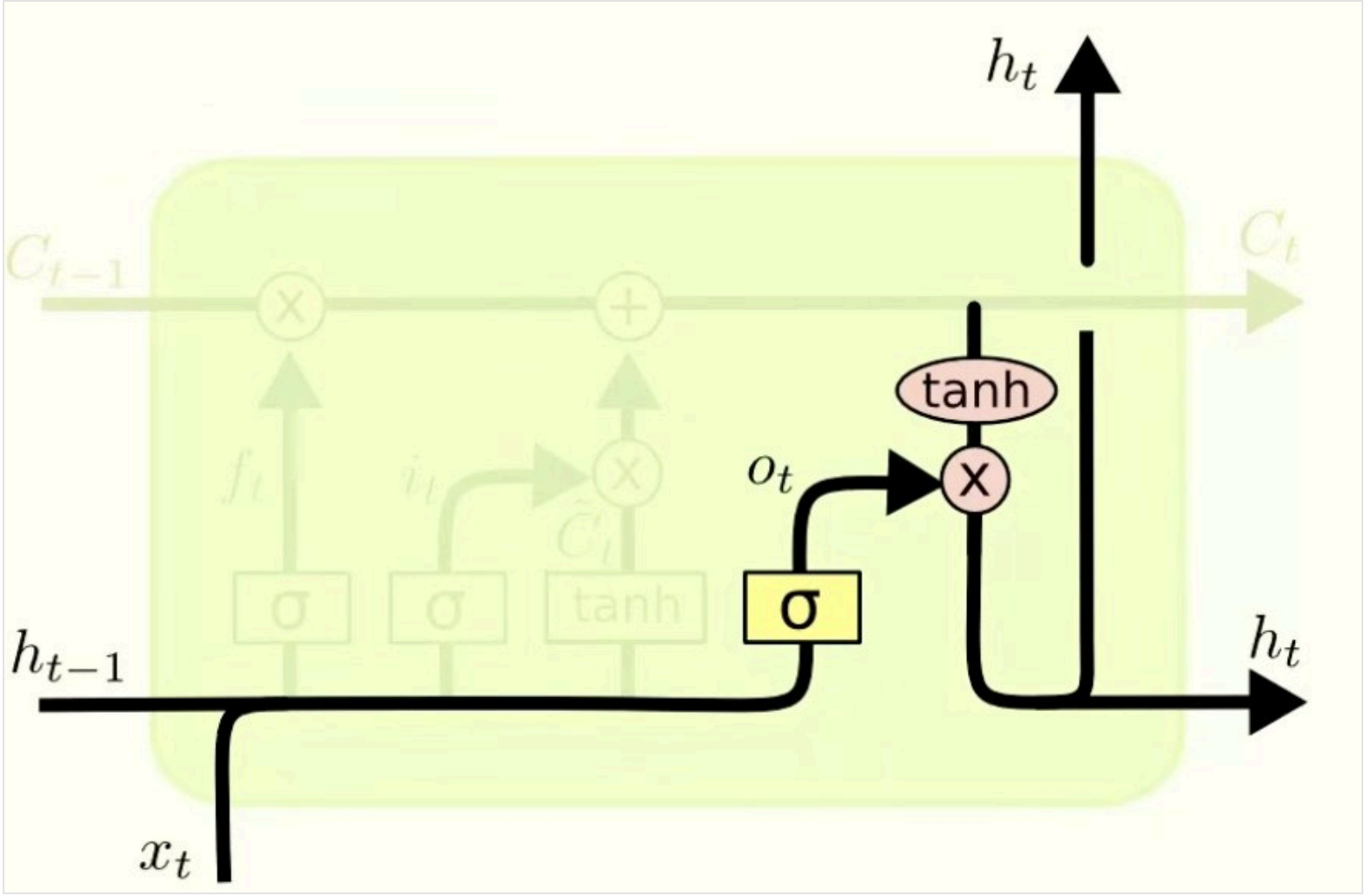
- 기억하고 싶으면 → 높은 값.
  잊고 싶으면 → 낮은 값
  → on/off 스위치 역할을 함.
  ⇒ 기억하고/잊을 것을 결정하는 역할



- $i_t = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i)$
  → which indices to write to
- $\tilde{C}_t = tanh(W_C \cdot [h_{t-1}, x_t] + b_C)$
  → what to write to those indices
- cell state에 추가할 새로운 정보를 결정



- 실제로 잊는 과정을 거침.
- $C_t = f_t * C_{t-1} + i_t * \tilde{C}_t$
  → 이전의 정보를 지우거나 유지하고 / 새로운 정보를 추가하는 과정

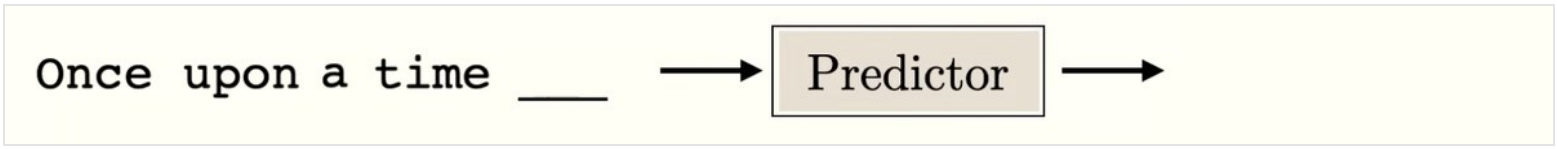


$$o_t = \sigma(W_o[h_{t-1}, x_t] + b_o)$$
$$h_t = o_t * tanh(C_t)$$

- 여기서는 어떤 것을 hidden state로 출력할지를 정하게 됨.
- transformer처럼? 어떤 것을 그대로 기억할 것인지를? 얻게 됨.

Sequence models

- Autoregressive models.



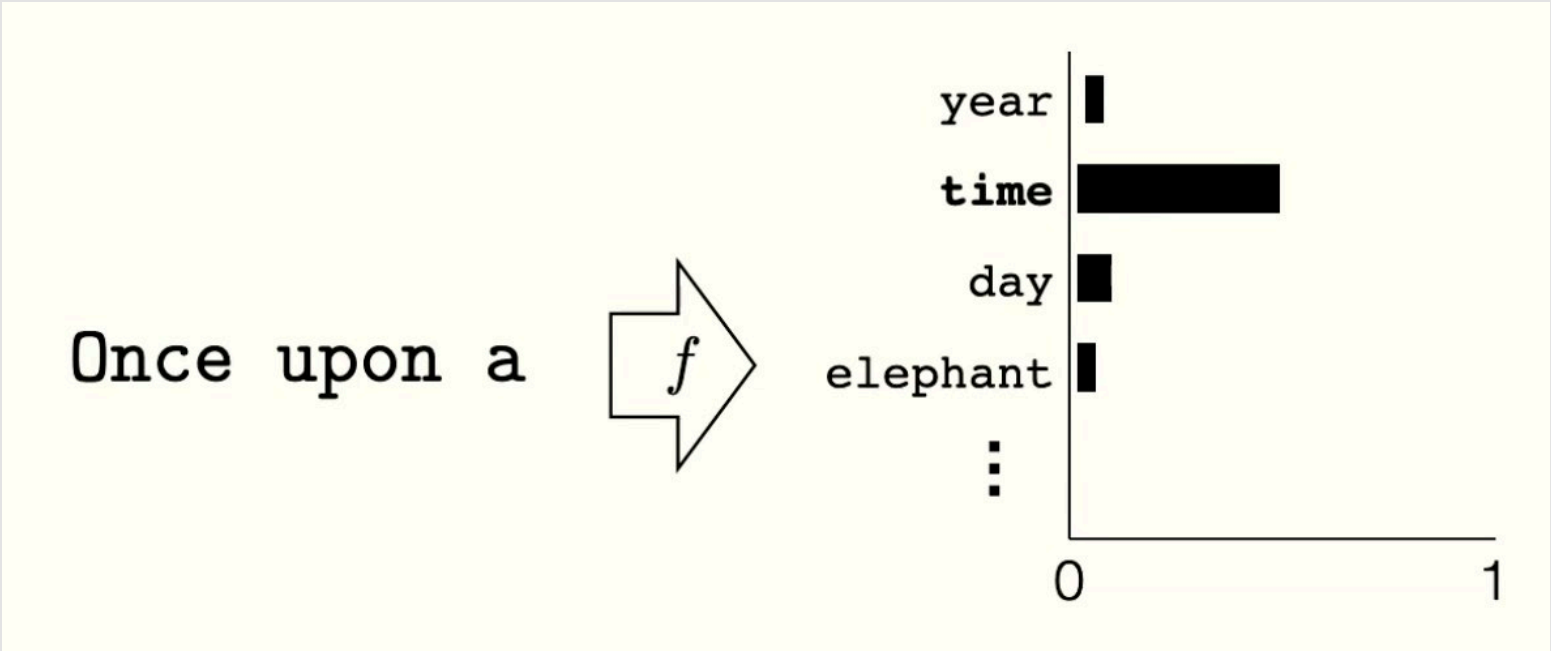
- → 다음 단어를 예측하는 방식.



- → 빈칸을 채우는 방식으로도 학습할 수 있음
- 문장 하나의 확률은 → 앞 단어의 조건부 확률의 연속으로 이루어짐.

Modeling a sequence of words

- → 그냥 **word classifier** 문제로 볼 수 있음



- softmax로 하나를 고르는 느낌으로다가.

- 근데 단어가 너무 많음 (K=100,000)
- 그래서 그냥, 단어 자체를 class로 두는 방법도 있음.
  → 훨씬 많이 예측해야 해서, 단어가 쌓일 수록 안 좋아질 수 있음.
- ⇒ 이 중간 정도 → byte 쌍 정도를 예측하는 게 좋음.